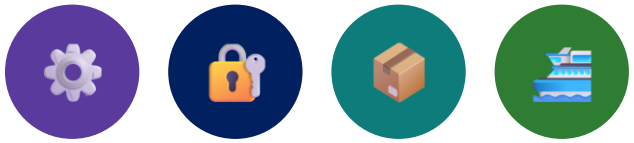
"A pipeline that quietly skips tests is not green — it is blind."

MODULE 43

GitHub Actions and CI/CD Integration

Build a reviewable GitHub Actions pipeline for the CRM backend and Angular frontend: triggers, caching, quality gates, secrets, and a package-once path to OpenShift.



TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 43 OVERVIEW

Build a GitHub Actions pipeline that verifies, gates, packages once, and protects secrets for the CRM backend and Angular frontend.

Pull requests need fast feedback while main and version tags need stronger gates — and build output must be traceable end to end, or staging quietly diverges from what CI verified.

DURATION & LAB

4 Hours Theory

Workflows, triggers, jobs/steps/actions/runners, caching, artifacts, gates, secrets, OIDC, and deployment.

2 Hours Lab

lab43-crm: a verify job, a quality gate, a package-once job with a checksum, and docs/ci-runbook.md.

Scenario

Verify the CRM backend serving Amina (CUS-1001) and Ravi (CUS-1002) with correlation lab-request-001.

MODULE ARC

Verify First

Every pull request runs checkout, a pinned JDK, and a real clean verify — never a skipped-test happy path.

Package Once

main and version tags package the JAR exactly once, checksum it, and tie it to the commit SHA.

Protect Secrets

Credentials live in scoped GitHub secrets or behind OIDC — never hardcoded in YAML or echoed to a log.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. No merge without a green verify job, a documented quality gate, a checksummed package artifact, and secrets kept out of Git.

WHY CI/CD MATTERS

Automating build, test, and release turns a fragile manual ritual into a repeatable, auditable pipeline.

Fast Feedback

Every push and pull request runs the same build and test suite automatically — broken code is caught in minutes, not at release time.

Fewer Manual Mistakes

Hand-run builds and copy-pasted deploy steps drift from what actually shipped — a pipeline runs the identical steps every time.

Consistent Environments

The same workflow that built the JAR for a pull request builds the JAR that ships to OpenShift — no 'works on my laptop' surprises.

Faster, Safer Releases

Small, frequently-integrated changes with automated gates are far cheaper to fix than a single large release assembled by hand.

KEY TAKEAWAY CI/CD replaces manual, error-prone build-and-deploy steps with a repeatable, auditable pipeline — that is the whole reason GitHub Actions exists in this course.

Why CI/CD Matters

CI/CD transforms how modern teams build, test, and deliver software—faster, safer, and with higher quality.

Deliver Faster
Automate build, test, and deployment to deliver features and fixes to users quickly and consistently.

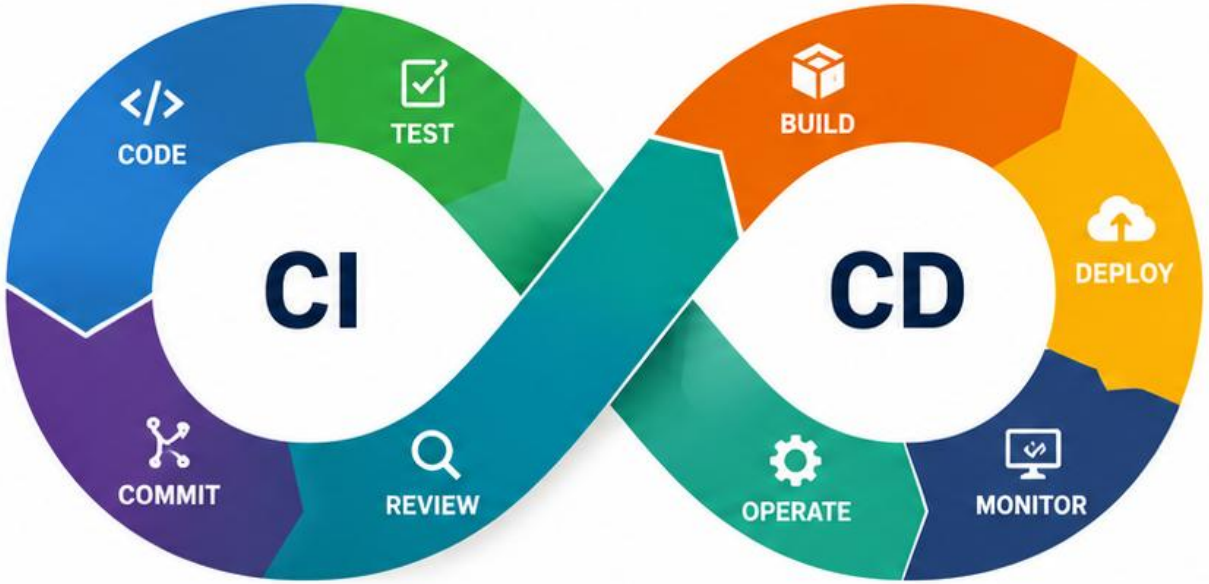
Improve Quality
Automated tests and quality gates catch issues early, reducing bugs in production.

Increase Collaboration
Standardized workflows and automation align teams across development, QA, and operations.

Reduce Manual Work
Eliminate repetitive tasks and manual deployments, freeing teams to focus on delivering value.

Ensure Consistency
Reliable, repeatable pipelines ensure every change follows the same proven process.

CI/CD Pipeline at a Glance



Continuous Integration (CI)
Developers integrate code frequently into a shared repository. Automated builds and tests validate changes early and often.

Continuous Delivery (CD)
Validated code is automatically built, tested, and deployed to environments with confidence and repeatability.

CI/CD is not just automation—it's a mindset of continuous improvement.

LEARNING OBJECTIVES

By the end of this module, you will be able to design, build, and secure a real GitHub Actions CI/CD pipeline.

Explain why CI/CD matters and how GitHub Actions models workflows, jobs, and steps.

Read and write workflow YAML: triggers, jobs, steps, runners, and actions.

Build and test a Java/Maven backend and an Angular frontend in GitHub Actions.

Use caching, artifacts, parallel jobs, job dependencies, and matrix builds.

Apply quality gates: CodeQL, dependency scanning, branch protection, required checks.

Manage GitHub Secrets, environment secrets, repository variables, and OIDC safely.

Build reusable workflows and composite actions, and containerize a build for OpenShift.

Diagnose a failing pipeline and apply enterprise pipeline best practices.

KEY TAKEAWAY These eight objectives map directly onto Lab 43's six exercises and the graded GitHub Actions CI/CD pipeline you will build and defend.

LEARNING OBJECTIVES

By the end of this module, you will be able to:

Understand CI/CD Concepts
Explain the purpose, benefits, and key components of Continuous Integration and Continuous Delivery.

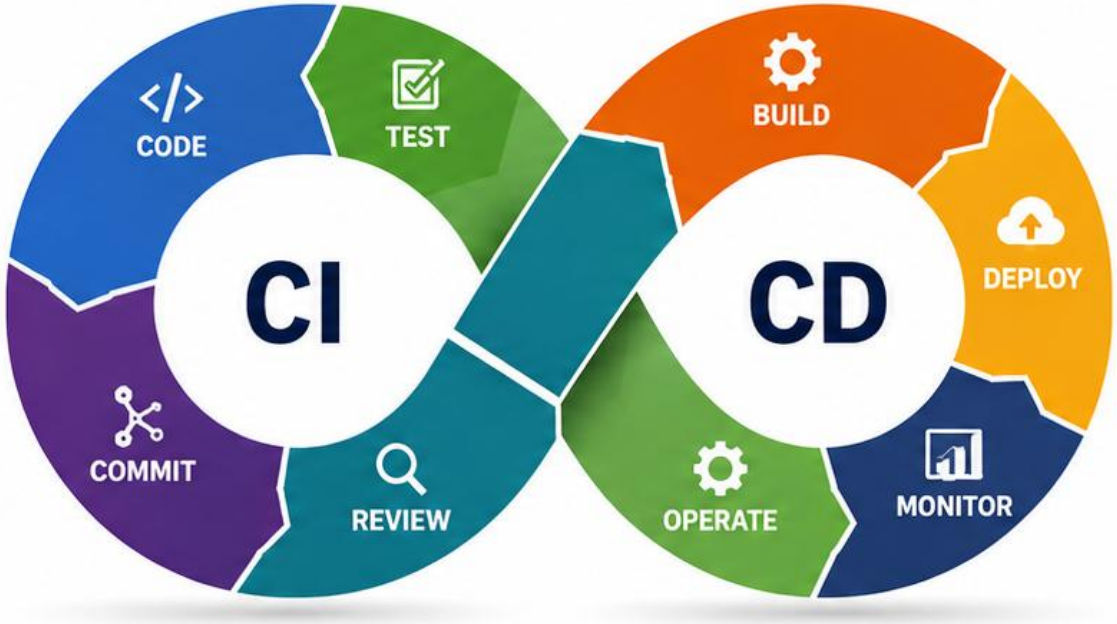
Work with GitHub Actions
Create and configure workflows triggered by events, jobs, steps, and actions.

Build and Test Applications
Automate Java (Maven) and Angular builds, run tests, and publish test reports.

Manage Artifacts and Caching
Upload and download artifacts and use caching to improve pipeline performance.

Implement Quality and Security Gates
Integrate code scanning, dependency checks, and branch protection to ensure code quality and security.

Automate Deployment
Build container images and deploy applications to OpenShift using secure, repeatable CI/CD pipelines.



You will gain the skills to build reliable, secure, and automated CI/CD pipelines that deliver value faster and with confidence.

CONTINUOUS INTEGRATION OVERVIEW

CI is a team practice — merge often, build and test automatically, and fail fast in front of the whole team.

Merge Often

Developers integrate code into a shared branch multiple times a day instead of isolating work on long-lived branches.

Build & Test Automatically

Every integration triggers an automated build and test run — humans do not manually verify each commit.

Fail Fast, Fix Fast

A red build is visible to the whole team immediately, so a broken commit gets fixed in minutes, not discovered days later.

KEY TAKEAWAY CI is a practice, not a tool — GitHub Actions is simply the engine this course uses to automate merge-often, build-and-test-automatically discipline.

CONTINUOUS INTEGRATION OVERVIEW

Continuous Integration (CI) is a development practice where developers integrate code changes frequently into a shared repository, and automated builds and tests validate those changes early and often.

KEY BENEFITS

Early Issue Detection
Find bugs and integration issues early in the development cycle.

Improved Code Quality
Automated testing and quality checks ensure consistent code standards.

Team Collaboration
Frequent integration reduces merge conflicts and keeps the codebase in a releasable state.

Higher Confidence
Every integrated change is verified through automation, increasing reliability.

Faster Feedback
Get quick feedback on changes to build, test, and improve continuously.

HOW CONTINUOUS INTEGRATION WORKS

CI IN THE SDLC

In short:
CI ensures that the code always remains in a working and tested state.

GITHUB ACTIONS OVERVIEW

Workflows live in your repository as versioned YAML and react to the events that already happen there.

Native to GitHub

Workflows live in the same repository as the code, versioned and reviewed exactly like application source.

Event-Driven

Workflows react to events — a push, a pull request, a schedule, or a manual trigger — rather than running on a fixed timer only.

Runs on Runners

Each job executes on a runner — a GitHub-hosted Ubuntu/Windows/macOS VM by default, or a self-hosted machine.

Marketplace Actions

Reusable building blocks — checkout, setup-java, upload-artifact — are shared as versioned Actions instead of hand-written scripts.

KEY TAKEAWAY GitHub Actions turns YAML committed alongside your code into the CI/CD engine — no separate pipeline tool or server to maintain.

GITHUB ACTIONS OVERVIEW

GitHub Actions is a powerful CI/CD platform built into GitHub. It enables you to automate your software workflows directly in your repository using simple YAML configuration.

WHY GITHUB ACTIONS?

Integrated with GitHub
Works seamlessly with repositories, issues, pull requests, and more.

Easy to Get Started
Use pre-built actions or create custom workflows with simple YAML.

Highly Extensible
Thousands of actions in the GitHub Marketplace to extend functionality.

Secure
Built with security in mind using secrets, permissions, and OIDC.

Cost-Effective
Generous free tier for public repositories and cost control for private repositories.

CORE CONCEPTS

Workflows
Automated processes defined in YAML files that run your jobs.

Events & Triggers
Workflows run when specific events occur, such as push, PR, schedule, or manual.

Jobs
A set of steps that run on the same runner. Jobs run in parallel by default.

Steps
Individual tasks executed in order within a job.

Actions
Reusable units of code that perform commonly used tasks.

Runners
Servers that run your workflows. GitHub-hosted or self-hosted.

HOW IT WORKS

1. Code Change
Developer pushes code or opens a pull request.

2. Trigger Event
GitHub detects the event and triggers the workflow.

3. Run Workflow
GitHub Actions executes the workflow steps on a runner.

4. Get Results
Results are reported back with success, failure, or feedback.

5. Take Action
Deploy, notify, or merge based on the results and conditions.

GitHub Actions helps teams build, test, and deploy with confidence—automatically.

GITHUB ACTIONS ARCHITECTURE

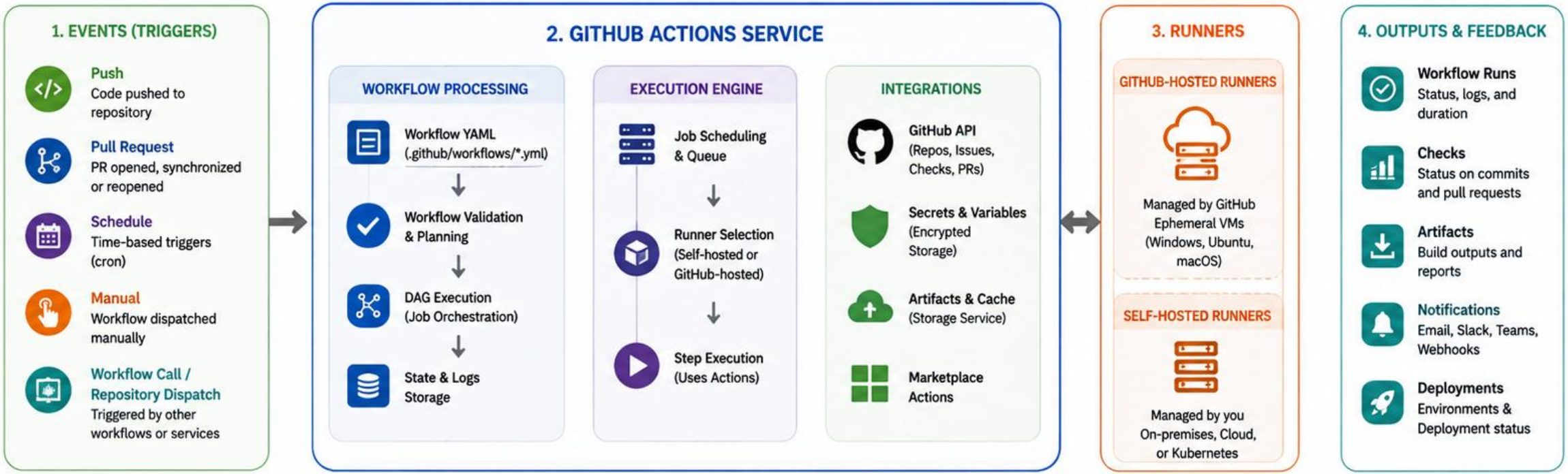
Event triggers a workflow; a workflow contains jobs; a job contains steps; steps run on a runner.

Concept	Role
Workflow	A YAML file in .github/workflows that defines one automated process.
Job	A group of steps that runs on one runner; jobs run in parallel by default.
Step	One task inside a job — either a shell command or a reusable action.
Runner	The VM (or container) that executes a job's steps.
Event	The trigger (push, pull_request, schedule, workflow_dispatch) that starts a run.
Artifact	A file or folder a job produces that later jobs or humans can download.

KEY TAKEAWAY Event triggers a workflow; a workflow contains jobs; a job contains steps; steps run on a runner — that chain is the architecture.

GITHUB ACTIONS ARCHITECTURE

GitHub Actions is a distributed, event-driven automation platform that runs your workflows in isolated environments and integrates deeply with your GitHub repositories.



GitHub Actions Architecture in a Nutshell

Events trigger workflows → GitHub Actions service processes and orchestrates → Runners execute jobs → Results and feedback are returned to GitHub and your tools.

WORKFLOWS

A workflow is a version-controlled YAML file with three required top-level keys: name, on, and jobs.

One File, One Workflow

.github/workflows/ci.yml (or any *.yml in that folder) is a complete, independent workflow definition.

Declarative YAML

name, on, and jobs are the three required top-level keys every workflow file needs.

Runs Are History

Every trigger creates a new 'run' visible in the Actions tab, with full logs for every job and step.

KEY TAKEAWAY A workflow is nothing more than a version-controlled YAML file that tells GitHub Actions what to do and when.

WORKFLOWS

A workflow is an automated process defined in a YAML file that builds, tests, and deploys your code.

KEY POINTS

Defined in YAML
Workflows are written in YAML files stored in the .github/workflows/ directory.

Event-Driven
Workflows run when specific events occur, such as push, pull request, schedule, or manual trigger.

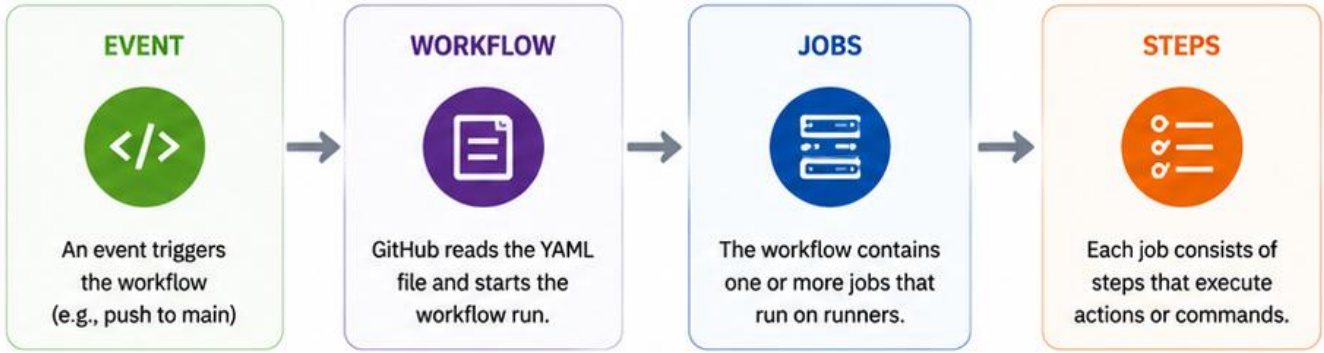
Composed of Jobs
A workflow contains one or more jobs that can run in parallel or sequentially.

Jobs Contain Steps
Each job is made up of a sequence of steps that execute actions or shell commands.

Reusable and Versioned
Workflows can be reused across repositories and versioned with your code.

TIP:
Think of a workflow as your CI/CD recipe—it defines when to run and what to do.

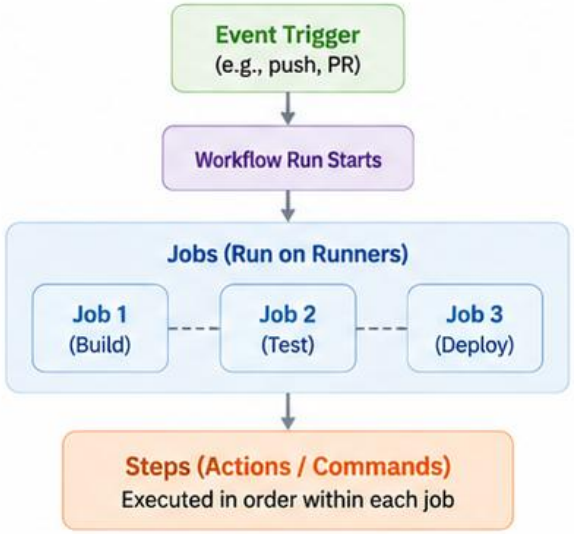
WORKFLOW ANATOMY



EXAMPLE WORKFLOW FILE

```yaml
name: CI Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          java-version: '17'
      - name: Build with Maven
        run: mvn -B clean verify
```

WORKFLOW STRUCTURE



COMMON TRIGGERS

push
Runs on code push to a branch.

pull_request
Runs when a PR is opened, synchronized, or reopened.

schedule
Runs on a cron-based schedule.

workflow_dispatch
Manually triggered from GitHub UI.

repository_dispatch
Triggered by external systems or workflows.

Workflows automate your entire development lifecycle—consistently, reliably, and repeatably.

EVENTS AND TRIGGERS

The on: key decides when a workflow runs at all — repository events, a schedule, or a human clicking a button.

Repository Events

push, pull_request, release, and issues are events GitHub itself raises as the repository changes.

Scheduled Events

schedule uses cron syntax to run a workflow at fixed times, independent of any code change.

Manual Events

workflow_dispatch lets a human trigger a run on demand from the Actions tab or the API.

KEY TAKEAWAY The on: key is the single decision that determines when a workflow runs at all — get it wrong and the whole pipeline is silently dead.

EVENTS AND TRIGGERS

GitHub Actions workflows run when specific events occur in your repository or on a schedule. These events act as triggers that start your automation.

KEY POINTS

Event-Driven Automation
Workflows start automatically when an event occurs in GitHub or an external system.

Flexible Triggers
Choose from many built-in events or custom triggers such as schedules or manual dispatch.

Fine-Grained Control
Use filters (branches, paths, tags, types) to run workflows only when needed.

Consistency and Efficiency
Triggers ensure workflows run consistently and only when relevant changes happen.

EXAMPLE: WORKFLOW TRIGGERED ON PUSH TO MAIN

```yaml
name: Build and Test
on:
  push:
    branches:
      - main
```

This workflow runs whenever code is pushed to the `main` branch.

COMMON EVENTS AND TRIGGERS

EVENT / TRIGGER	ICON	DESCRIPTION	TYPICAL USE CASE
push		Triggered when code is pushed to a branch.	Build and test on every code push.
pull_request		Triggered when a pull request is opened, synchronized, or reopened.	Run tests and checks on PRs.
schedule		Triggered on a cron schedule.	Nightly builds, cleanup jobs, or periodic tasks.
workflow_dispatch		Triggered manually from the GitHub UI or API.	Run workflows on demand.
repository_dispatch		Triggered by an external system sending a webhook.	Integrate with external tools or systems.
release		Triggered when a release is published.	Build and publish release artifacts.
workflow_call		Triggered when another workflow calls this workflow.	Reusable workflows across repositories.

EVENT FLOW

Code Change → **Event Occurs** → **Workflow Triggered** → **Jobs Execute** → **Results Reported**

Code Change
Developer makes a change and pushes or triggers an event.

Event Occurs
GitHub detects the event.

Workflow Triggered
The workflow file matching the event is triggered.

Jobs Execute
Jobs and steps run on GitHub-hosted or self-hosted runners.

Results Reported
Status, logs, and artifacts are available in GitHub.

JOBS, STEPS, ACTIONS, AND RUNNERS

The four nouns every workflow file is built from — learn this vocabulary before reading any real YAML.

Jobs

A named unit of work with its own runner; independent jobs execute in parallel unless one depends on another.

Steps

Ordered tasks inside a job — run a shell command with run:, or invoke a reusable action with uses:.

Actions

Packaged, versioned units of automation — actions/checkout@v4, actions/setup-java@v4 — referenced by owner/repo@ref.

Runners

GitHub-hosted runners (ubuntu-latest, windows-latest, macos-latest) or self-hosted machines that execute a job's steps.

KEY TAKEAWAY Jobs contain steps; steps run shell commands or actions; runners are the machines that make it all happen — this is the vocabulary for every workflow in this module.

JOBS

A job is a set of steps that runs on the same runner. Workflows can have one or more jobs that run sequentially or in parallel.

KEY POINTS

Group of Steps
A job contains a sequence of steps that execute actions or shell commands.

Runs on a Runner
All steps in a job run on the same runner environment.

Parallel or Sequential
Jobs can run in parallel by default or in sequence using dependencies.

Isolated Environment
Each job runs in a clean, isolated environment, ensuring consistency.

Share Data with Artifacts
Jobs can share data using artifacts or caches.

JOB ANATOMY

JOB CHARACTERISTICS

- All steps in a job share the same workspace.
- Environment variables are scoped to the job.
- If any step fails, the job fails.
- Jobs can produce outputs and artifacts.
- Jobs can depend on other jobs using needs.

EXAMPLE YAML

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '17'
      - run: mvn clean package
  test:
    runs-on: ubuntu-latest
    needs: build
    steps:
    ... (additional steps)
```

EXAMPLE: MULTI-JOB WORKFLOW

COMMON USES OF JOBS

Build
Compile and package code

Test
Run automated tests

Security Scan
Scan code and dependencies

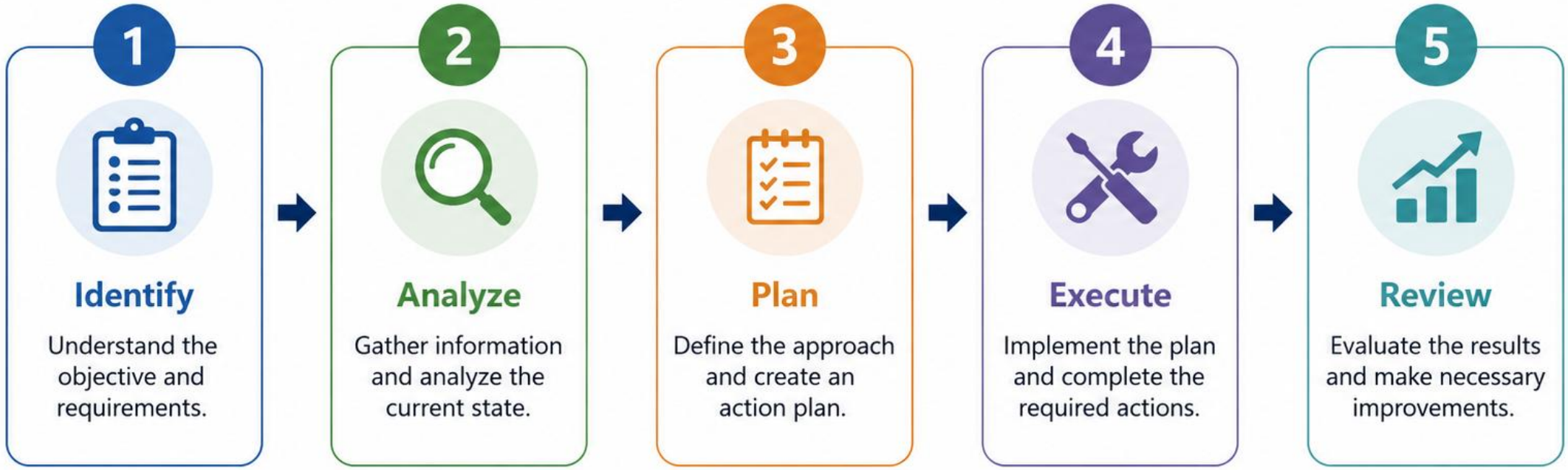
Publish
Publish artifacts or packages

Deploy
Deploy to staging or production

Jobs are the building blocks of your workflow.

Steps

A sequence of ordered actions to achieve a goal or complete a process.



Key Takeaway: Following clear steps ensures consistency, reduces errors, and helps you achieve better outcomes.

Stay Focused | Follow in Order | Ensure Quality | Improve Continuously

ACTIONS

Actions are specific operations performed to achieve a task or objective.

CREATE

Create something new or add information.

UPDATE

Modify existing information or make changes.

DELETE

Remove information or resources that are no longer needed.

VIEW

Retrieve and view information or details.

SHARE

Distribute or share information with others.

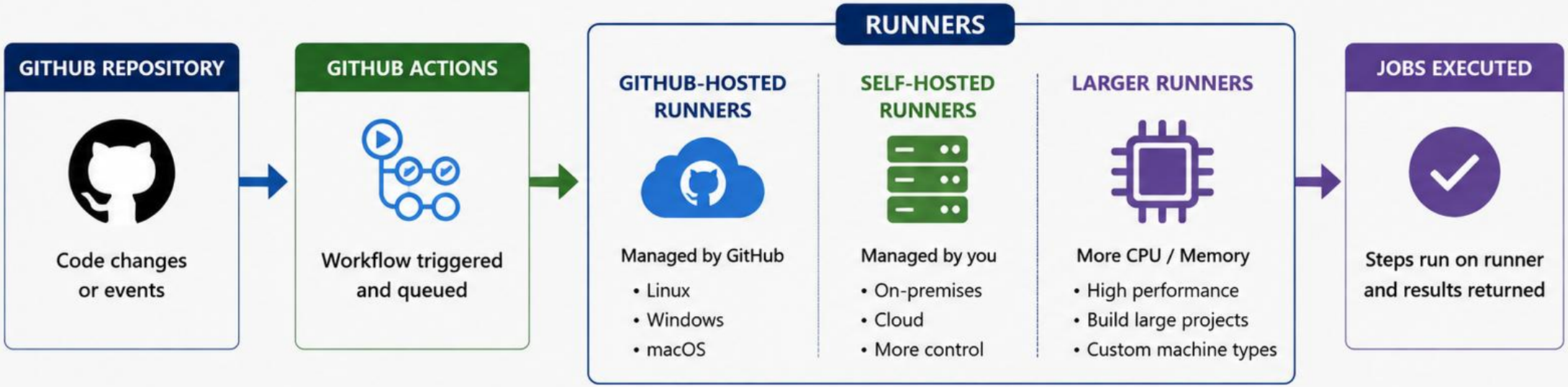
AUTOMATE

Perform tasks automatically to save time and reduce errors.

Key Takeaway: Choosing the right action and performing it consistently leads to efficiency, accuracy, and better outcomes.

RUNNERS

Runners are the servers that execute your GitHub Actions workflows.



Execute Jobs
Runners pick up jobs from the queue and run workflow steps.

Isolated Environment
Each job runs in a clean, isolated environment to ensure consistency.

Secure
GitHub-hosted runners are ephemeral and secure by default.

Scalable
Automatically scales to handle your CI/CD workload.

Key Takeaway: Runners provide the compute power and environments needed to build, test, and deploy your application reliably and at scale.

WORKFLOW YAML STRUCTURE

Every workflow reduces to the same three required keys, refined with good habits from real-world pipelines.

REQUIRED TOP-LEVEL KEYS

- name: a human-readable label shown in the Actions tab.
- on: the event(s) that trigger this workflow.
- jobs: one or more named jobs, each with runs-on and steps.

GOOD HABITS

- Pin actions to a major version tag, e.g. actions/checkout@v4.
- Give jobs and steps clear name: values — logs are read under pressure.
- Keep one workflow file per concern (ci.yml, cd.yml) rather than one giant file.

Minimal workflow skeleton

```
name: CRM CI
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "build here"
```

KEY TAKEAWAY name, on, and jobs are the only required keys — everything else, caching, artifacts, matrices, is refinement on top of this skeleton.

WORKFLOW YAML STRUCTURE

A GitHub Actions workflow is defined in a YAML file that describes when to run, what to run, and how to run it.

STRUCTURE ELEMENTS

1. name
Name of the workflow

2. on
Defines events that trigger the workflow

3. jobs
Collection of jobs that run in the workflow

4. steps
Sequence of tasks executed in a job

5. uses / run
Run an action or execute a shell command

.github/workflows/ci.yml

```yaml
name: Java CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK
        uses: actions/setup-java@v4
          with:
            java-version: '17'
      - name: Build with Maven
        run: mvn -B clean verify
```

1. name
Human-readable name for the workflow.

2. on
Events that trigger the workflow execution.

3. jobs
Defines one or more jobs to be executed.

4. steps
Ordered list of tasks executed in the job.

5. uses / run
Execute a reusable action or run a shell command.

Key Takeaway: A workflow YAML file defines the trigger, jobs, and steps for automation in a clear and repeatable way.

.github/workflows DIRECTORY

GitHub Actions only ever looks in one folder — get the path wrong and the workflow is silently ignored.

Fixed Location

.github/workflows/ is the only folder GitHub Actions scans — a workflow YAML anywhere else is silently ignored.

One File Per Workflow

ci.yml, cd.yml, codeql.yml can coexist; each *.yml file in the folder is its own independent workflow.

Reviewed Like Code

Workflow changes go through the same pull-request review as application code — a pipeline change is a code change.

KEY TAKEAWAY If a workflow is not saved under .github/workflows/ with a .yml or .yaml extension, GitHub Actions never sees it — check the path first when a workflow “doesn't run.”

.github/workflows DIRECTORY

The **.github/workflows** directory is where GitHub Actions looks for workflow YAML files.
Each YAML file defines one automation workflow.

REPOSITORY STRUCTURE

- my-app (repository root)
  - .github
    - workflows
      - ci.yml
      - build.yml
      - test.yml
      - deploy.yml
  - src
  - tests
  - docs
  - README.md

Default Location

GitHub Actions automatically detects workflow files in **.github/workflows**.

Multiple Workflows

You can have multiple YAML files. Each file represents a separate workflow.

Version Controlled

Workflow files are stored in your repository and versioned with your code.

Automatically Discovered

When pushed or triggered, GitHub Actions reads the YAML files and runs the jobs.

EXAMPLE WORKFLOW FILES

ci.yml (Continuous Integration)

build.yml (Build Process)

test.yml (Run Tests)

deploy.yml (Deployment)

EXAMPLE: ci.yml

```yaml
name: CI Pipeline
on: [ push, pull_request ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up JDK
```

Key Takeaway: Place workflow YAML files in **.github/workflows** to automate your build, test, and deployment processes directly from your repository.

on: TRIGGERS

Five trigger types cover almost everything a CI/CD pipeline in this course needs to react to.

Trigger	What Starts the Run
push	A commit lands on a branch (optionally filtered by branches: or paths:).
pull_request	A PR is opened, synchronized, or reopened against a target branch.
workflow_dispatch	A human clicks Run workflow in the Actions tab, optionally passing inputs.
schedule	A cron expression fires at a fixed UTC time, with no code change involved.
release	A GitHub Release is published, often used to trigger a deployment workflow.

KEY TAKEAWAY Combine triggers deliberately — pull_request for fast feedback, push to main for stronger gates, and tags/release for anything that ships.

`on:` TRIGGERS

The `on:` key defines the events that trigger a GitHub Actions workflow to run.

WHAT ARE TRIGGERS?

Triggers specify when your workflow should start. GitHub Actions listens for these events and starts the workflow automatically.

Repository Events

Events like push, pull_request, or release in your repository.

Scheduled Events

Run workflows at specific intervals using a cron schedule.

Manual Triggers

Start workflows manually from the GitHub UI.

Workflow Calls

Trigger a workflow from another workflow using `workflow_call`.

External Events

Trigger workflows from external services using `repository_dispatch`.

EXAMPLE: on: SECTION

```yaml
name: CI Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 2 * * *'
  workflow_dispatch:
  workflow_call:
  repository_dispatch:
    types: [ custom_event ]
```

COMMON on: TRIGGERS

Trigger	Purpose	Example
push	Run on code push to branches	`push: branches: [ main ]`
pull_request	Run on pull requests	`pull_request: branches: [ main ]`
schedule	Run on a scheduled time (cron)	`schedule: - cron: '0 2 * * *'`
workflow_dispatch	Manual trigger from GitHub UI	`workflow_dispatch:`
workflow_call	Called by another workflow	`workflow_call:`
repository_dispatch	Triggered by external events	`repository_dispatch: types: [ custom_event ]`

Key Takeaway: Use the `on:` key to define when your workflow runs. Choose the right trigger to automate your CI/CD processes efficiently.

PUSH, PULL REQUEST, MANUAL, AND SCHEDULED TRIGGERS

The same on: key, four different shapes — each earns a different amount of pipeline rigor.

Push Trigger

on: push: branches: [main] runs verify on every commit that lands on main — the strongest, most trusted gate.

Pull Request Trigger

on: pull_request runs the same checks before merge, giving reviewers a green/red signal directly on the PR.

Manual Dispatch

on: workflow_dispatch adds a Run workflow button — ideal for an on-demand deploy or a one-off maintenance job.

Scheduled Workflows

on: schedule: - cron: '0 3 * * *' runs a workflow every night regardless of whether anyone pushed code.

KEY TAKEAWAY PRs get fast verify-only feedback; main and tags earn stronger gates; deploy is opt-in via dispatch or a version tag — never automatic on every PR.

PUSH TRIGGER

Runs a GitHub Actions workflow automatically when code is pushed to the repository.

What It Does

Starts the workflow when changes are pushed to a branch.

When It Runs

On **git push** (new commits) to the configured branch(es).

Common Use Cases

- Build and test on every code change
- Continuous Integration
- Quality checks before merge

Key Benefits

- Fast feedback on code changes
- Automates the CI process
- Keeps main branches stable

Example: Push Trigger Workflow

```yaml
name: CI Pipeline

on:
  push:
    branches:
      - main
      - develop

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Build and test
        run: mvn clean verify
```

Push Trigger Flow

Key Takeaway: Use the **push** trigger to run CI workflows automatically whenever code is pushed, ensuring continuous integration and early issue detection.

PULL REQUEST TRIGGER

Runs a GitHub Actions workflow when a pull request is created, updated, or synchronized.

What It Does

Starts the workflow when a pull request event occurs in the repository.

When It Runs

On pull_request events such as opened, **synchronize** (new commits), reopened, or closed.

Common Use Cases

- Run tests on PRs
- Code quality and security scans
- Validate before merge
- Preview builds and environments

Key Benefits

- Catch issues early in the review process
- Ensure code quality and compliance
- Provide feedback before merging

Example: Pull Request Trigger Workflow

```yaml
name: PR Validation Pipeline

on:
  pull_request:
    types: [ opened, synchronize, reopened, closed ]
    branches: [ main, develop ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Run tests
        run: mvn test

```

Pull Request Trigger Flow

Key Takeaway: Use the `pull_request` trigger to validate code changes through pull requests, ensuring quality, security, and reliability before code is merged.

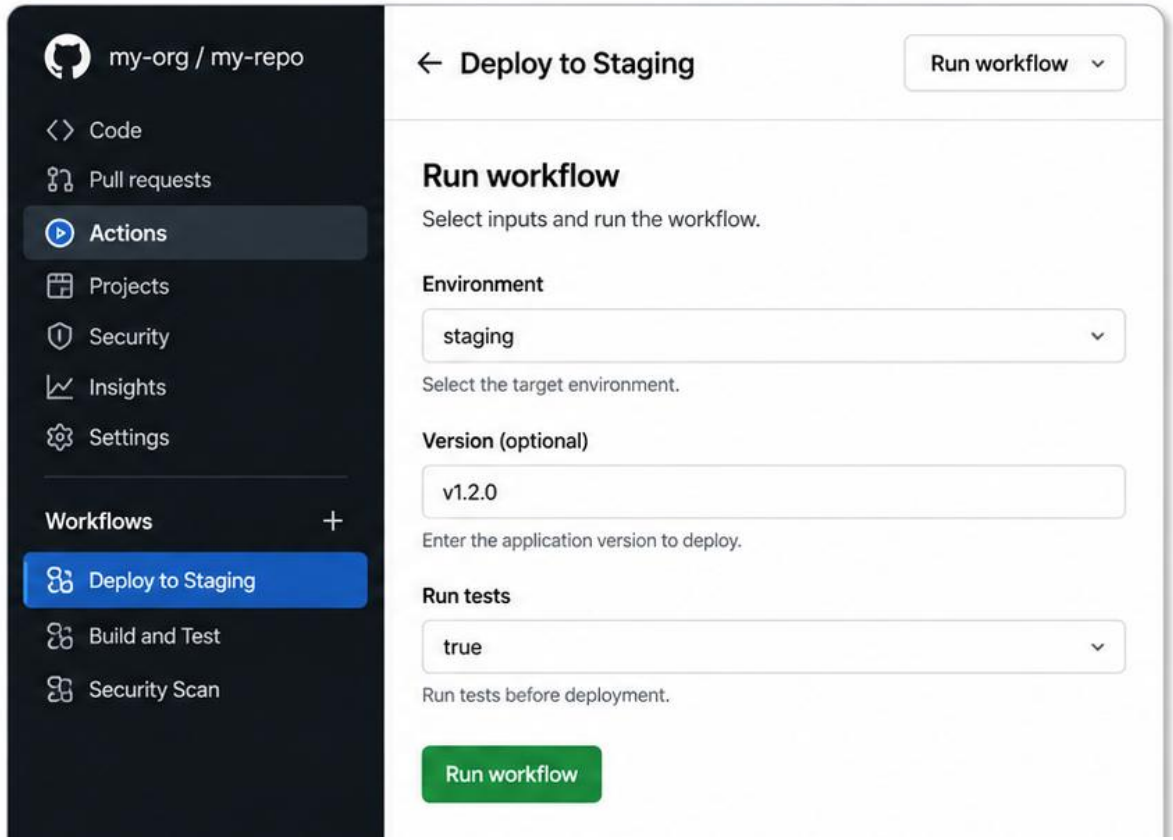
Manual Workflow Dispatch

GitHub Actions

Manual Workflow Dispatch allows you to run a GitHub Actions workflow **on demand**.
It is useful for ad-hoc tasks, re-runs with different parameters, or environment promotions.

Key Points

- Triggered manually from the **GitHub Actions UI**.
- Defined using the `workflow_dispatch` event in YAML.
- Supports **input parameters** for flexible execution.
- Ideal for **re-running** failed pipelines or releasing specific versions.
- **Does not** require a code push or pull request event.
- Provides **full visibility** in the Actions tab like other runs.



Key Takeaway

Manual Workflow Dispatch gives you **control** to run workflows whenever you need—without waiting for a code event.

Scheduled workflows allow you to run GitHub Actions automatically at a **defined time or interval** using a cron expression.

Key Points

- Run workflows automatically based on a **cron schedule**.
- Useful for nightly builds, data sync, housekeeping, and reporting tasks.
- Configured using the `schedule` event in YAML.
- Supports **UTC** time zone.
- Can define one or **multiple schedules**.
- Visible in the Actions tab like other workflow runs.
- Helps keep systems **up-to-date** without manual triggers.

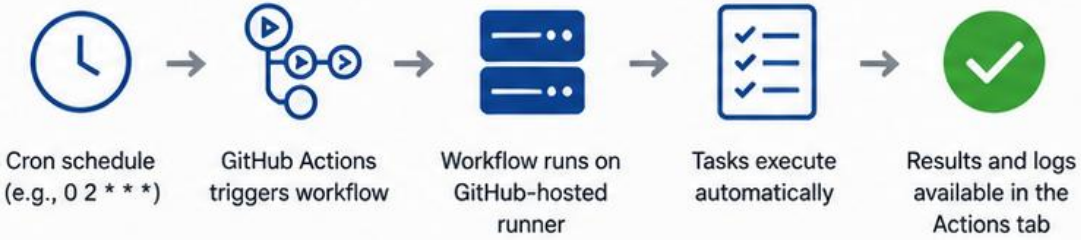
Key Takeaway

Scheduled Workflows help automate recurring tasks reliably and consistently using **cron-based scheduling**.

Example: .github/workflows/nightly-build.yml

```yaml
name: Nightly Build
on:
  schedule:
    # Run every day at 02:00 UTC
    - cron: '0 2 * * *'
    # Run every Sunday at 03:30 UTC
    - cron: '30 3 * * 0'
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build with Maven
        run: mvn -B clean package
```

How Scheduled Workflows Work



TRY IT YOURSELF — EXERCISE 1: DEFINE PIPELINE TRIGGERS

Reinforces: deciding what runs on pull_request, main push, and version tags before writing any YAML.

STEPS (notes/lab43-pipeline-policy.md)

Step	What To Do
1 -- Matrix	Fill a table: event -> jobs (verify always; package on main/tags; deploy later/not yet).
2 -- Check the Reference	PRs get fast feedback; main/tags get stronger gates; deploy creds never in Git.
3 -- CRM Identity	Note synthetic fixtures may appear only in test evidence (CUS-1001, lab-request-001).
4 -- Scope	Pre-lab only -- do not finish the full graded lab in this exercise.

Reference policy

```
pull_request: verify only
push main: verify + package
tag v*: verify + manual deploy
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-pipeline-policy.md (workspace: examples/module-43-exercises).

JAVA BUILD WORKFLOW

Checkout, pin the JDK, then clean verify — the same three steps the CRM's own verify job runs.

WHAT THE VERIFY JOB DOES

- Checks out the repository with actions/checkout@v4.
- Installs a pinned JDK with actions/setup-java@v4.
- Runs the Maven Wrapper's clean verify goal, never a partial build.

WHY IT MATTERS

- The exact JDK version is pinned — no “works with 17, fails with 21” surprises.
- clean verify compiles, runs tests, and packages — nothing is skipped silently.
- A red verify job blocks merge before a broken commit reaches main.

CRM verify job (from Lab 43)

```
verify:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: "21"
        cache: maven
    - run: ./mvnw -B -ntp clean verify
```

KEY TAKEAWAY A Java build workflow is checkout, pin the JDK, then clean verify — the same three steps this module's own CRM pipeline runs.

A Java Build Workflow automates the **build**, **test**, and **verification** of a Java application using Maven and GitHub Actions.

Key Points

- Triggers on code **push** or **pull request** events.
- Sets up Java environment with the desired **JDK version**.
- Uses **Maven** to build the project.
- Runs unit and integration **tests**.
- Generates and publishes **test reports**.
- Stores build **artifacts** for later use or deployment.
- Enforces quality and provides **fast feedback** to developers.

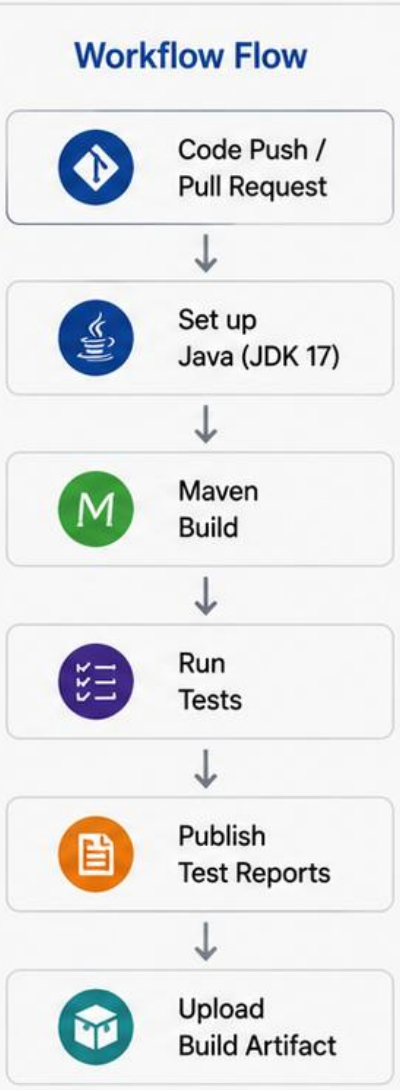
Key Takeaway

A Java Build Workflow ensures **code quality**, **reliability**, and **consistency** by automating the build and test process in CI.

Example: .github/workflows/java-build.yml

```yaml
name: Java Build Workflow
on:
  push:
    branches: [ main, develop ]
  pull.request:
    branches: [ main, develop ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Cache Maven packages
        uses: actions/cache@v4
        with:
          path: ~/.m2/repository
          key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
          restore-keys: ${{ runner.os }}-maven-
      - name: Build with Maven
        run: mvn -B clean verify
      - name: Upload Test Reports
        uses: actions/upload-artifact@v4
        with:
          name: test-reports
          path: target/surefire-reports
      - name: Upload Build Artifact
        uses: actions/upload-artifact@v4
        with:
          name: app-jar
          path: target/*.jar
```

Workflow Flow



TRY IT YOURSELF — EXERCISE 2: FILL ci.yml TODOs

Reinforces: filling in workflow triggers, jobs, and step TODOs for the CRM's own CI contract.

STEPS (notes/lab43-workflow-todos.md)

Step	What To Do
1 -- Skeleton	Create a ci.yml skeleton with blanks for java-version, verify command, artifact paths, package if:.
2 -- Fill	Fill blanks using Temurin 21 and ./mvnw -B clean verify / package-once pattern.
3 -- Secrets Comment	Add a YAML comment: '# secrets via GitHub Actions secrets -- never hardcode'.
4 -- Scope	State that pushing and greening the workflow is Lab 43, not this exercise.

ci.yml skeleton (excerpt)

```
on: {pull_request, push: {branches: [main], tags: ['v*']}}
verify: setup-java 21 -> ./mvnw -B clean verify
package: needs verify, if main/tags -> checksum + upload
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-workflow-todos.md (workspace: examples/module-43-exercises).

MAVEN BUILD IN GITHUB ACTIONS

./mvnw -B -ntp clean verify is the one command this course's Java workflows are built around.

Prefer the Wrapper

./mvnw guarantees the exact Maven version the project was built with — no drift between a developer's laptop and the runner.

Batch Mode

-B (batch mode) and -ntp (no transfer progress) keep CI logs clean and non-interactive.

clean verify, Not Just compile

verify runs the full lifecycle including tests and packaging — compile alone proves almost nothing.

KEY TAKEAWAY ./mvnw -B -ntp clean verify is the one command this course's Java workflows build around — memorize it.

GitHub Actions uses Maven to **compile, test, package,** and verify your Java application automatically in the pipeline.

Key Points

- Uses the official **Maven action** for consistent builds.
- Automatically downloads and **caches dependencies**.
- Performs **clean build** for a reproducible build.
- Runs unit and integration **tests**.
- Honors **settings.xml** and custom Maven profiles.
- Generates build **artifacts** (JAR/WAR).
- Supports **custom goals** and options.
- Works with **Maven Wrapper** (mvnw) or installed Maven.

Key Takeaway

Maven in GitHub Actions ensures a **consistent, automated,** and **reliable** build process for Java applications.

Example: .github/workflows/maven-build.yml

```yaml
name: Maven Build
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]
jobs:
 build:
   runs-on: ubuntu-latest
   steps:
     - name: Checkout code
       uses: actions/checkout@v4
     - name: Set up JDK 17
       uses: actions/setup-java@v4
       with:
         distribution: 'temurin'
         java-version: '17'
         cache: 'maven'
     - name: Build with Maven
       run: mvn -B clean package
     - name: Upload build artifact
       uses: actions/upload-artifact@v4
       with:
         name: app-jar
         path: target/*.jar
```

Maven Build Process

- **Checkout Code** — Get source from repository
- **Set Up JDK** — Install and configure JDK
- **Resolve Dependencies** — Download and cache dependencies
- **Maven Build** — mvn clean package
- **Run Tests** — Execute unit and integration tests
- **Build Artifact** — Create and store JAR/WAR

Common Maven Goals Used

clean	compile	test	package	verify	install

DEPENDENCY CACHING

cache: maven is one line, and it is the single biggest lever on pipeline speed for a Maven build.

Setting	Effect
cache: maven (in setup-java)	Automatically caches ~/.m2/repository keyed on pom.xml hashes.
Cache hit	Dependency resolution skips network downloads — builds finish in a fraction of the time.
Cache miss (first run, or pom.xml changed)	A full download happens once, then the cache is rewritten for next time.
Stale key symptom	Cache "never hits" — usually a key or path mismatch; keep the default setup-java behavior unless there's a reason to override it.

KEY TAKEAWAY Dependency caching is one line — cache: maven — but it is the single biggest lever on pipeline speed for a Maven project.

Dependency caching stores downloaded dependencies between workflow runs to **speed up builds** and reduce external network calls.

Key Points

- Caches dependencies to **reduce build time**.
- Uses a **cache key** to uniquely identify cache entries.
- **Restores** cache before dependency download.
- **Saves cache** after successful dependency resolution.
- Automatically invalidates when dependencies change.
- Greatly **improves CI performance** and developer productivity.
- Works with Maven, npm, Gradle, and more.

Key Takeaway

Caching dependencies ensures **faster**, **more reliable**, and **cost-efficient** builds by avoiding repeated downloads.

Example: Caching Maven dependencies

```yaml
- name: Cache Maven packages
  uses: actions/cache@v4
  with:
    path: ~/.m2/repository
    key: ${{ runner.os }}-maven-${{ hashFiles('**/pom.xml') }}
    restore-keys: ${{ runner.os }}-maven-

- name: Build with Maven
  run: mvn -B clean verify
```

How Dependency Caching Works

1. **Restore Cache** — Check for an existing cache using the key.
2. **Use Cache (Hit)** — If found, restore dependencies.
3. **Build & Resolve** — Run the build and resolve dependencies.
4. **Save Cache (Miss)** — If no cache found, save for future runs.

Benefits

Faster Builds
Reduces build time by reusing downloaded dependencies.

Lower Network Usage
Minimizes external downloads and bandwidth consumption.

More Reliable CI
Less dependency on external repositories availability.

Cost Efficient
Saves CI minutes and infrastructure costs.

Automatic & Seamless
Handled transparently within your workflows.

RUNNING JUNIT TESTS

Tests already run inside clean verify — and skipping them to force a green build is a named anti-pattern.

Tests Run Inside verify

mvn verify already runs Surefire (unit) and Failsafe (integration) tests — no separate 'test step' is required.

Never -DskipTests on the Default Path

Skipping tests to force a green build is explicitly a false-green anti-pattern this module names directly.

Fail the Job on Red

A failing test fails the step, which fails the job, which blocks the pull request — that chain is the entire point.

KEY TAKEAWAY If a workflow needs -DskipTests to pass, the workflow isn't broken — the tests are, and that is what verify exists to catch.

JUnit tests automatically **validate your Java code** for correctness and reliability during the CI pipeline. GitHub Actions executes all unit and integration tests and reports the results.

Key Points

- Uses **Maven Surefire Plugin** to run JUnit tests.
- Executes tests during the CI build pipeline.
- Supports **JUnit 4** and **JUnit 5**.
- Generates **JUnit XML** test reports.
- Build **fails** automatically on test failures.
- Captures test logs and results for **visibility**.
- Enforces **code quality** and prevents regressions.
- Works with **unit** and **integration** tests.

Key Takeaway

Running JUnit tests in GitHub Actions ensures **high-quality code**, catches issues early, and builds confidence in every commit.

Example: Run JUnit Tests in GitHub Actions

```yaml
- name: Run JUnit Tests
  run: mvn test

- name: Publish JUnit Test Results
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: junit-test-results
    path: target/surefire-reports/*.xml
```

How JUnit Tests Run in GitHub Actions

1. **Checkout Code** — Get source code from the repository.
2. **Set Up Java** — Install and configure the required JDK.
3. **Run Tests** — Execute 'mvn test' to run JUnit tests.
4. **Generate Reports** — JUnit XML reports are generated.
5. **Upload Results** — Test results are uploaded as workflow artifacts.

On Failure: The workflow is marked as failed and the build stops.

Benefits

Early Defect Detection
Finds issues before code reaches production.

Improved Code Quality
Encourages clean, maintainable, and testable code.

Reliable Builds
Ensures only tested and working code is delivered.

Detailed Test Reports
JUnit XML reports provide detailed test outcomes.

Team Confidence
Builds trust in every change with automated testing.

TRY IT YOURSELF — EXERCISE 3: PLAN JDK 21 VERIFY JOB

Reinforces: planning setup-java 21, Maven caching, and a real clean verify without skipTests.

STEPS (notes/lab43-java21-verify.md)

Step	What To Do
1 -- Setup	List Actions steps: checkout, setup-java Temurin 21 with Maven cache, ./mvnw -B clean verify.
2 -- Check the Reference	Upload Surefire/Failsafe reports even on failure (if: always()).
3 -- Failure Drill Plan	Write how you will intentionally break one test, observe CI red, then restore.
4 -- Local Habit	Note local preflight: java -version shows 21; ./mvnw -v before pushing.

Local preflight

```
java -version   # must show 21
./mvnw -v
./mvnw -B -ntp clean verify
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-java21-verify.md (workspace: examples/module-43-exercises).

PUBLISHING TEST REPORTS

if: always() on the upload step is what turns a failed pipeline into diagnosable evidence instead of a dead end.

WHY UPLOAD REPORTS

- Surefire/Failsafe XML and HTML reports explain *why* a build went red, not just that it did.
- A reviewer or instructor can download the artifact without re-running the build.

THE if: always() RULE

- upload-artifact must run even when verify fails — that is exactly when reports matter most.
- Without if: always(), a failed step skips every step after it, including the upload.

Upload reports even on failure

```
- name: Upload test reports
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-reports
    path: |
      **/target/surefire-reports/**
      **/target/failsafe-reports/**
```

KEY TAKEAWAY if: always() on the upload step is what turns a failed pipeline into diagnosable evidence instead of a dead end.

Publishing test reports makes test results **visible** in GitHub, improves **feedback**, and helps track **quality** over time.

Key Points

- JUnit tests generate **XML reports** (Surefire format).
- GitHub Actions can publish these reports **automatically**.
- Reports appear in the **Actions UI** under "Tests".
- Helps quickly identify **failing tests**.
- Supports **trend analysis** and historical tracking.
- Improves **transparency** and team collaboration.
- Easy to configure using **built-in actions**.

Key Takeaway

Publishing test reports provides clear visibility into test outcomes and helps teams deliver high-quality code faster.

Example: Publish JUnit Test Results

```yaml
- name: Run Tests
  run: mvn test

- name: Publish JUnit Test Results
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: junit-test-results
    path: target/surefire-reports/*.xml

- name: Publish Test Report to GitHub
  uses: dorny/test-reporter@v1
  if: always()
  with:
    name: JUnit Tests
    path: target/surefire-reports/*.xml
    reporter: java-junit
    fail-on-error: false
```

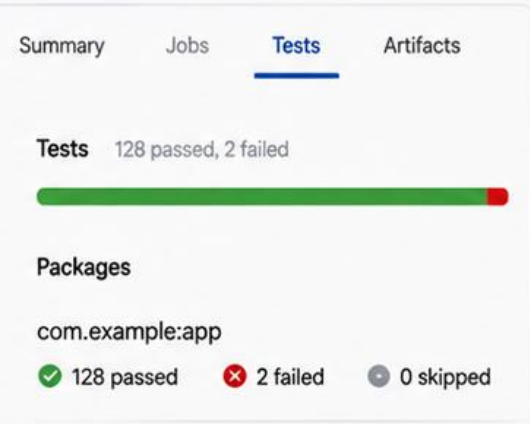
How Test Reports Are Published

1. **Run Tests** — Maven runs JUnit tests and generates XML reports.
2. **Generate Reports** — Reports are created in target/surefire-reports/.
3. **Upload Artifact** — XML report files are uploaded as workflow artifacts.
4. **Publish Report** — Test Reporter action parses and publishes results.
5. **View Results** — Results are displayed in the GitHub Actions Tests tab.

Where Reports Appear

After the workflow run completes:

1. Go to Actions tab
2. Select the workflow run
3. Click on the Summary
4. View results in the Tests section



Best Practices

- Always publish reports
- Use consistent report
- Publish both artifacts and
- Keep test names clear
- Monitor trends to improve

BUILD ARTIFACTS

If a job's output isn't uploaded as an artifact, it vanishes the instant the runner is torn down.

What an Artifact Is

Any file or folder a job produces — a JAR, a test report, a coverage file — that outlives the runner's disk.

Scoped to a Run

Artifacts belong to the workflow run that created them and are retained for a configurable number of days.

Not the Same as a Release Asset

An artifact is CI evidence; a Release asset is a durable, versioned download — don't confuse the two.

KEY TAKEAWAY If a job's output isn't uploaded as an artifact, it disappears the moment the runner is torn down — nothing survives by default.

Build artifacts are files produced during the CI build that are **preserved** and **shared** between workflow jobs or for **download** and **deployment**.

Key Points

- Artifacts store **compiled binaries**, **packages**, **reports**, and other build outputs.
- **Persisted** beyond the job that created them.
- Can be **shared** between jobs or workflows.
- Essential for **multi-job pipelines** and **deployments**.
- Easy to **upload** and **download** using built-in actions.
- Artifacts are **retained** for a configurable period.
- Improve **traceability** and reproducibility.

Key Takeaway

Auild artifacts ensure that important build outputs are **preserved**, **shareable**, and **ready** for deployment or further validation.

Example: Create and Upload Build Artifacts

```yaml
- name: Build Application
  run: mvn -B clean package

- name: Upload Build Artifact
  uses: actions/upload-artifact@v4
  with:
    name: app-jar
    path: target/*.jar
    retention-days: 7

- name: Upload Test Reports
  uses: actions/upload-artifact@v4
  with:
    name: surefire-reports
    path: target/surefire-reports/
```

How Build Artifacts Work

1. **Build** — Application is compiled and produces outputs.
2. **Create Artifact** — Selected files/folders are packaged as an artifact.
3. **Upload** — Artifact is uploaded to GitHub Actions storage.
4. **Download** — Artifacts can be downloaded in later jobs or manually.
5. **Use** — Used for testing, analysis, or deployment.

Common Use Cases

Share Between Jobs
Pass build outputs to downstream jobs.

Store Test Reports
Publish test results, coverage, and logs.

Deployment Packages
Store JAR/WAR/ZIP files for deployments.

Debug & Traceability
Keep artifacts for troubleshooting and audit purposes.

Versioned Outputs
Track and compare artifacts across workflow runs.

Best Practices ✓ Use meaningful ✓ Upload only necessary ✓ Set appropriate ✓ Organize artifacts by ✓ Secure sensitive files

UPLOADING AND DOWNLOADING ARTIFACTS

Upload in the job that builds it, download by name in the job that needs it.

Uploading Artifacts

actions/upload-artifact@v4 with a name: and path: saves files from the current job — GitHub retains them 90 days by default (configurable per repository).

Downloading Artifacts

actions/download-artifact@v4 in a later job pulls a named artifact back down by exact name — this is how package hands the JAR to deploy; if-no-files-found: warn avoids a false failure.

KEY TAKEAWAY Upload in the job that builds it, download by name in the job that needs it — artifacts are the only supported way to pass files between jobs.

Uploading Artifacts

Artifacts allow you to persist files generated during a job so they can be downloaded or used in later jobs or workflows.

What are Artifacts?
Files or directories saved from a workflow run and stored by GitHub.

Why Upload Artifacts?
Share build outputs, logs, test reports, coverage, or any important files across jobs or for download.

Where are Artifacts Used?
Can be downloaded manually from the Actions UI or consumed by later jobs using `needs` or `download-artifact`.

Retention
Artifacts are retained for up to 90 days by default. This can be configured in repository settings.

Best Practices
- Upload only what is necessary.
- Use meaningful artifact names.
- Don't store secrets or sensitive data.

Artifacts help make your pipeline observable, traceable, and easy to debug.

Artifact Flow in GitHub Actions

1. Build/Test
Compile code, run tests, generate reports or outputs

2. Upload Artifact
Use actions/upload-artifact to save files from the job

3. Stored by GitHub
Artifact is stored securely and associated with the workflow run

4. Download / Use
Download manually from the UI or use in later jobs and workflows

Example: Uploading Artifacts in GitHub Actions

```yaml
- name: Upload Test Results
  uses: actions/upload-artifact@v4
  with:
    name: test-results                    # Name of the artifact
    path: target/surefire-reports/        # Path to the file or directory
    if-no-files-found: warn               # Warn instead of failing if not found
    retention-days: 30                    # How long to retain the artifact
```

Artifacts provide visibility, traceability, and repeatability across your CI/CD pipeline.

DOWNLOADING ARTIFACTS

Artifacts uploaded by earlier jobs can be downloaded and used in later jobs or manually from the GitHub Actions UI.

What is Downloading?
Retrieving artifacts that were previously uploaded in any job or workflow run.

Why Download Artifacts?
Reuse build outputs, test reports, logs, binaries, or any files generated in previous jobs.

How It Works
Use the `actions/download-artifact` action in your workflow to download files to the runner.

Where It Can Be Used
In later jobs of the same workflow or even in separate reusable workflows.

Retention
Artifacts are retained for up to 90 days by default (unless changed in repository settings).

Artifacts help you pass data between jobs and preserve important outputs.

Artifact Download Flow in GitHub Actions

1. Earlier Job
A previous job uploads the artifact.

2. Stored by GitHub
Artifact is stored securely with the workflow run.

3. Download in Later Job
Use `actions/download-artifact` to retrieve the artifact.

4. Available on Runner
Artifact files are downloaded to the runner workspace.

5. Use in Workflow
Files are used for testing, deployment, or further steps.

Example: Downloading Artifacts in GitHub Actions

```yaml
- name: Download Test Results
  uses: actions/download-artifact@v4
  with:
    name: test-results                    # Name of the artifact to download
    path: ./downloaded-artifacts          # Destination directory on the runner
    if-no-files-found: warn               # Warn if artifact is not found
```

Downloading artifacts ensures outputs from earlier jobs are available for later stages in your CI/CD pipeline.

TRY IT YOURSELF — EXERCISE 4: PACKAGE-ONCE IDENTITY

Reinforces: sketching a package-once JAR plus SHA-256 checksum tied to the commit for later promotion.

STEPS (notes/lab43-immutable-jar.md)

Step	What To Do
1 -- Steps	Outline: package once, write SHA256SUMS, record GITHUB_SHA, upload artifact.
2 -- Check the Reference	Lab 44 promotes this identity -- rebuilding silently on the deploy agent breaks the chain.
3 -- Example Lines	Draft example checksum file lines (fake hashes OK) including the commit id.
4 -- Anti-Pattern	Name one anti-pattern: packaging differently in deploy than in CI.

SHA256SUMS (example)

```
3f9a...c21e  crm-1.0.0.jar
commit=8f0c2ab
run=482
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-immutable-jar.md (workspace: examples/module-43-exercises).

PARALLEL JOBS AND JOB DEPENDENCIES

Independent jobs run in parallel for speed; needs: expresses the one ordering rule that actually matters.

Parallel Jobs

Jobs with no needs: run concurrently on separate runners by default — lint, unit tests, and a scan can all run at once.

Job Dependencies

needs: verify makes package wait for verify to succeed first — the DAG this creates is what enforces 'test before you package.'

KEY TAKEAWAY Independent jobs run in parallel for speed; needs: expresses the one ordering rule that actually matters — never build twice.

PARALLEL JOBS

Parallel jobs allow multiple jobs in a workflow to run at the same time. This reduces total pipeline execution time and improves developer productivity.

What are Parallel Jobs?

Multiple jobs that have no dependencies can run concurrently on separate runners.

Why Use Parallel Jobs?

Faster feedback, better resource utilization, and shorter end-to-end pipeline duration.

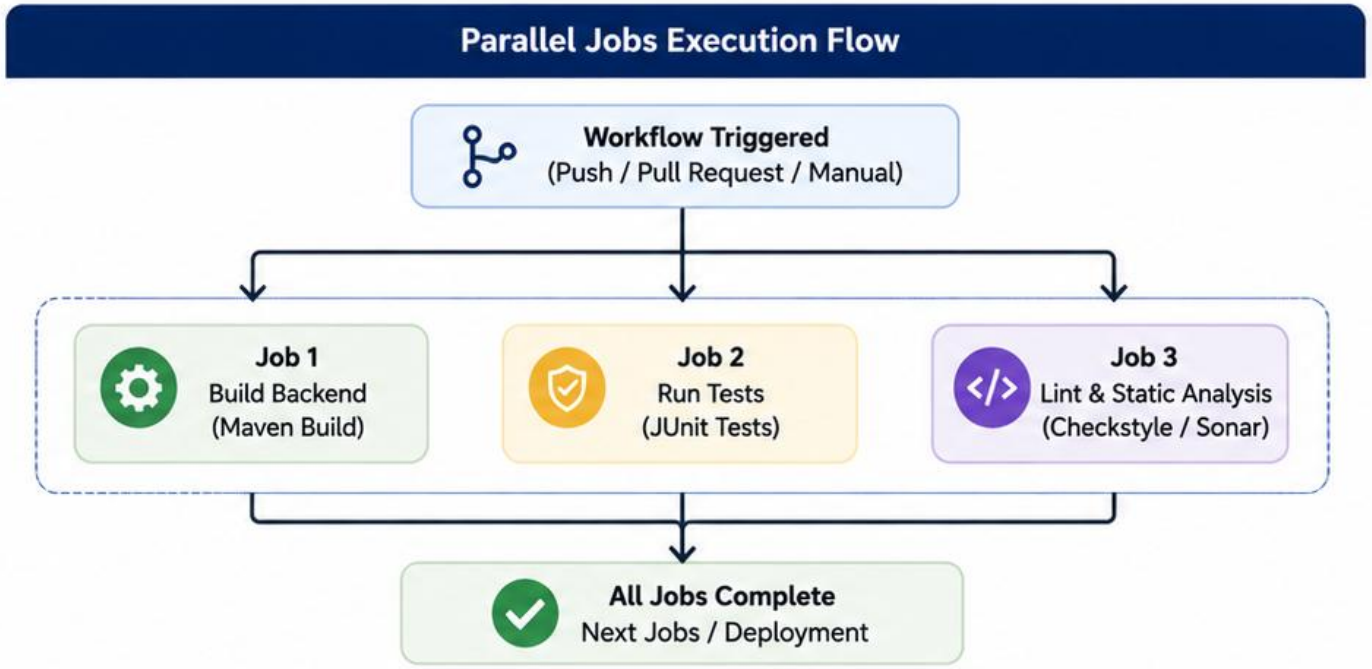
How It Works

Jobs without dependencies start at the same time. GitHub Actions schedules them on available runners.

Best Practices

- Run independent tasks in parallel.
- Use job dependencies only when required.
- Keep jobs small and focused.
- Use caching to speed up builds.

Parallel jobs shorten feedback loops and make your CI/CD pipeline more efficient.



Example: Parallel Jobs in GitHub Actions

```yaml
jobs:
  build-backend:
    runs-on: ubuntu-latest                # Runs in parallel with build-backend and test
    steps: [ ... ]
  test:
    runs-on: ubuntu-latest                # Runs in parallel with build-backend and test
    steps: [ ... ]
  lint:
    runs-on: ubuntu-latest                # Runs in parallel with build-backend and test
    steps: [ ... ]
```

JOB DEPENDENCIES

Job dependencies define the order in which jobs run in a workflow. A job can start only after the jobs it depends on have completed successfully.

What are Job Dependencies?

Dependencies control the execution sequence between jobs using the needs keyword.

Why Use Job Dependencies?

Ensure a logical flow, prevent failures, and promote reliable pipeline execution.

How It Works

Specify job IDs in needs. The dependent job waits for all listed jobs to complete successfully.

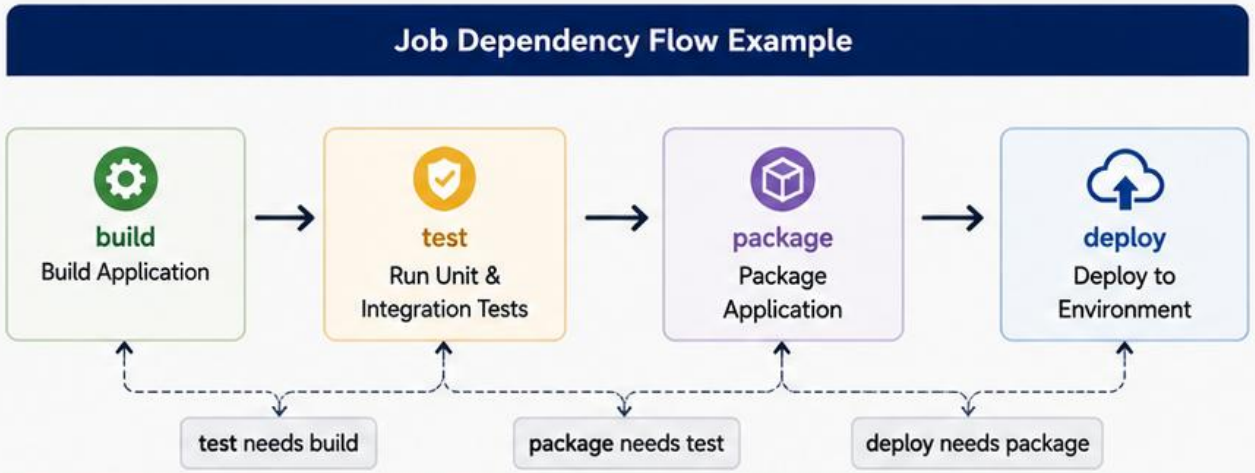
Key Points

- If any required job fails, dependent jobs are skipped.
- Use dependencies to enforce build → test → deploy flow.
- Dependencies can be sequential or fan-out/fan-in.

Best Practices

- Keep dependency chains clear and minimal.
- Run independent jobs in parallel.
- Use job outputs to pass data between dependent jobs.

Proper job dependencies create a reliable and predictable CI/CD pipeline



Example: Job Dependencies in GitHub Actions

```yaml
jobs:
  build:
    runs-on: ubuntu-latest                    # First job - no dependencies
    steps: [ ... ]
  test:
    needs: build                              # Runs after build succeeds
    runs-on: ubuntu-latest
    steps: [ ... ]
  package:
    needs: test                               # Runs after test succeeds
    runs-on: ubuntu-latest
    steps: [ ... ]
  deploy:
    needs: package                            # Runs after package succeeds
    runs-on: ubuntu-latest
    steps: [ ... ]
```

Use job dependencies to enforce order, reduce risk, and ensure your pipeline runs smoothly from build to deployment

MATRIX BUILDS

A matrix multiplies one job definition across many environments to prove real compatibility.

WHY A MATRIX

- Runs the same job multiple times with different inputs — Java versions, OS, or Node versions.
- Catches “works on 21, breaks on 17” or “works on Linux, breaks on Windows” before a user does.

MATRIX MECHANICS

- strategy.matrix defines the axes; GitHub Actions expands every combination automatically.
- fail-fast: false lets every combination finish and report, instead of canceling siblings on first failure.

Matrix across two JDKs

```
strategy:
  matrix:
    java: [17, 21]
steps:
  - uses: actions/setup-java@v4
    with: { java-version: "${{ matrix.java }}" }
```

KEY TAKEAWAY A matrix multiplies one job definition across many environments — use it to prove compatibility, not to hide a flaky test.

MATRIX BUILDS

Matrix builds allow you to run the same job multiple times with different combinations of values (e.g., OS, Java versions, Node versions). This helps ensure compatibility across environments.

What are Matrix Builds?

Run the same job multiple times across a matrix of configuration values.

Why Use Matrix Builds?

Test across multiple environments, versions, and configurations to catch issues early and increase confidence in your builds.

How It Works

Define variables inside the `matrix` section. GitHub Actions creates a job for each combination.

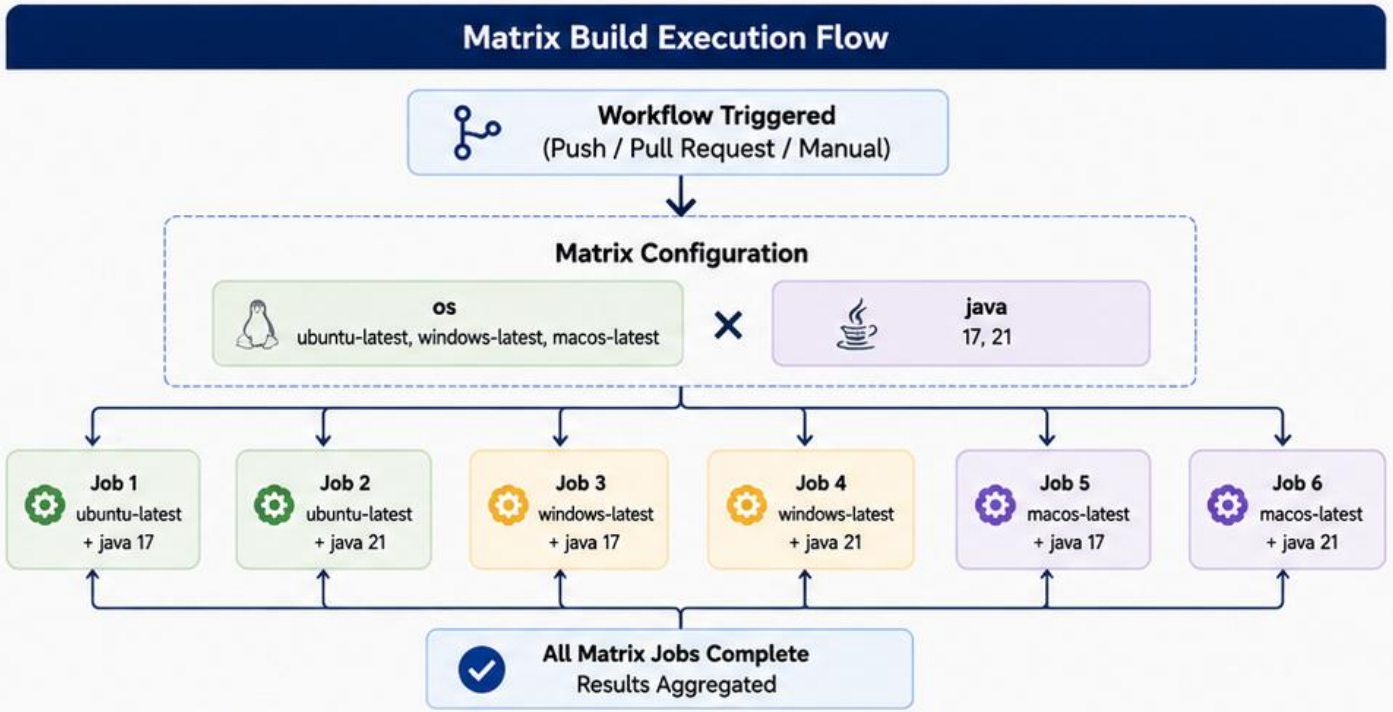
Common Use Cases

- Cross-OS builds (ubuntu, windows, macos)
- Multiple language/runtime versions
- Different Node.js versions
- Database versions, browsers, etc.

Benefits

- Improves compatibility and reliability
- Detects environment-specific issues
- Saves time by running in parallel

Matrix builds help you test your code in multiple



Example: Matrix Build in GitHub Actions

```yaml
jobs:
  build:
    runs-on: ${{ matrix.os }}
    strategy:                                    # Runs on each OS in parallel
      matrix:                                    # Combinations: 3 OS × 2 Java = 6 jobs
        os: [ubuntu-latest,windows-latest,macos-latest]
        java: [17, 21]
  steps:
    - uses:actions/checkout@v4
    - name:Set up JDK ${{ matrix.java }}
      uses:actions/setup-java@v4
      with:                                      # Installs the specified Java version
        distribution: temurin
        java-version: ${{ matrix.java }}

                                                 # Builds and runs tests
    - uame:Build with Maven
```

Key Takeaways

- Matrix builds run the same job with different combinations.
- Jobs run in parallel for faster feedback.
- Ideal for testing across OS, language versions, and tools.
- Keep the matrix focused to manage build time and cost.

BUILDING ANGULAR APPLICATIONS

The Angular half of this pipeline mirrors the Java half exactly — pin the runtime, cache, build strictly.

Same Repo, Same Pipeline

A full-stack pipeline builds the Angular frontend and the Spring Boot backend as separate jobs in one workflow.

actions/setup-node

Pins a specific Node.js LTS version, exactly like setup-java pins the JDK — reproducibility applies to both stacks.

npm Cache Built In

setup-node's cache: 'npm' caches ~/.npm the same way cache: maven caches ~/.m2.

KEY TAKEAWAY The Angular half of this pipeline mirrors the Java half exactly — pin the runtime, cache dependencies, run a strict build.

BUILDING ANGULAR APPLICATIONS

Angular applications are built using the Angular CLI. The production build compiles TypeScript, optimizes assets, and outputs a deployable bundle.

What Happens During Build?

Angular compiles TypeScript, processes templates, bundles JavaScript, optimizes assets, and generates static files for deployment.

Why Build in CI?

Ensures consistent builds, catches errors early, and produces optimized artifacts for environments.

Build Output

The output is a production-ready application in the `dist/` folder (default).

Key Benefits

- Smaller bundle size (tree-shaking, minification)
- Faster load times (asset optimization)
- Consistent across all environments
- Ready for deployment (static files)

Best Practices

- Always use production builds for deployment.
- Use environment configurations.
- Run tests before building.
- Cache node_modules to speed up builds.

A production build ensures your Angular application is fast, optimized, and ready for deployment.

Angular Build Process

1. Write Code
Develop Angular application using TypeScript.

2. Angular CLI
`ng build` command starts the build process.

3. Compile & Bundle
TypeScript is compiled, modules are bundled, and assets processed.

4. Optimize
Minification, tree-shaking, and optimization reduce bundle size.

5. Output
Production-ready files generated in the `dist/` folder.

Example: Building Angular in GitHub Actions

```yaml
- name: Setup Node.js
  uses: actions/setup-node@v4
  with:                                    # Install the required Node.js version
    node-version: '20'                     # Enable npm cache
    cache: 'npm'

- name: Install Dependencies               # Clean install dependencies
  run: npm ci

- name: Build Angular Application          # Run Angular production build
  run: npm run build -- --configuration=production   # Uses environment configuration

- name: Upload Build Artifact
  uses: actions/upload-artifact@v4         # Upload dist/ folder as an artifact
  with:                                    # for use in later jobs
    name: angular-dist
    path: dist/
```

Build Command

```
ng build --configuration=production
```

Common Options

- `--configuration=production`
  Optimized for production
- `--configuration=development`
  Faster builds with source maps
- `--output-path=dist/custom`
  Custom output directory
- `--base-href=/my-app/`
  Set base URL for deployment

NPM INSTALL AND BUILD

npm ci, not npm install, is the CI-safe install command — it enforces the lockfile instead of trusting it.

THE TWO COMMANDS

- npm ci installs exactly what package-lock.json specifies — no surprise upgrades, unlike npm install.
- ng build --configuration production (via npm run build) produces the optimized, deployable dist/ bundle.

WHY npm ci, NOT npm install

- npm ci deletes node_modules first and fails if package.json and the lockfile disagree.
- That strictness is exactly what a CI runner should demand — a local "it installed fine" is not proof.

Angular build job

```
frontend-build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: "20", cache: "npm" }
    - run: npm ci
    - run: npm run build -- --configuration production
```

KEY TAKEAWAY npm ci, not npm install, is the CI-safe install command — it enforces the lockfile instead of trusting it.

NPM INSTALL AND BUILD

npm is the Node Package Manager for JavaScript. It installs project dependencies and runs the build to produce an optimized Angular application.

What Happens?

`npm install` downloads and installs all dependencies defined in `package.json`.

Why It Matters in CI?

Ensures all developers and the CI environment use the same dependency versions for consistent builds.

Build Step

`npm run build` compiles TypeScript, optimizes assets, and outputs the production-ready files to `dist/`.

Key Benefits

- Reproducible builds
- Faster builds with caching
- Optimized and minified output
- Ready for deployment

Best Practices

- Use `npm ci` for clean and reliable installs in CI.
- Cache node_modules to speed up builds.
- Always build in production mode.
- Do not commit node_modules.

A clean install and optimized build ensure your Angular application is reliable, fast, and production-ready.

npm Install and Build Process

1. Install Dependencies
`npm install` or `npm ci` installs packages from package.json.

2. Resolve Dependencies
All required libraries are downloaded and placed in `node_modules/`.

3. Build Application
`npm run build` compiles TypeScript and processes assets.

4. Optimize Output
Code is minified, tree-shaken, and assets are optimized for production.

5. Output Ready
Production-ready files are generated in the `dist/` folder.

Example: npm Install and Build in GitHub Actions

```yaml
- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm'                                  # Enable npm cache

- name: Install Dependencies (clean install)
  run: npm ci                                     # Faster and reliable install

- name: Build Angular Application
  run: npm run build -- --configuration=production    # Production build

- name: Upload Angular Build Artifact
  uses: actions/upload-artifact@v4
  with:
    name: angular-dist
    path: dist/                                   # Upload dist/ for later stages
```

Common Commands

```
npm install      # Install dependencies
npm ci           # Clean install (CI)
npm run build    # Build for development
npm run build -- --configuration=production
                 # Build for production
```

Outputs

`dist/` Production-ready Angular app (HTML, CSS, JS, assets)

Tips

- Use `npm ci` in CI for faster, deterministic installs.
- Cache npm dependencies to reduce build time.
- Always build with `--configuration=production`

ANGULAR TESTS IN CI

Any Angular test command in CI must be explicitly non-interactive and headless, or the job never finishes.

Headless by Default

ng test --watch=false --browsers=ChromeHeadless runs Karma/Jasmine unit tests without a visible browser or a hanging watcher.

CI Must Not Watch

The default ng test watches for file changes and never exits — that hangs a runner until it times out.

Angular's Current Test Runner

Newer Angular CLI versions default to a built-in test runner (Vitest-based) — the same --watch=false discipline still applies.

KEY TAKEAWAY Any Angular test command in CI must be explicitly non-interactive and headless, or the job never finishes.

ANGULAR TESTS IN CI

Automated Angular tests in CI ensure that your frontend code is reliable, maintainable, and regression-free.

What are Angular Tests?

Angular tests are automated tests written using frameworks like Karma and Jasmine to verify components, services, and application behavior.

Why Run Tests in CI?

Catches bugs early, prevents regressions, and ensures consistent quality across all branches and pull requests.

Types of Angular Tests

- Unit Tests – Components, services, pipes
- Integration Tests – Modules, routing
- End-to-End Tests – User flows (with Cypress)

Benefits

- Fast feedback for developers
- Higher code coverage
- Confident releases
- Improved application stability

Automated Angular tests in CI help deliver high-quality applications with confidence.

Angular Tests in CI Workflow

Example: Angular Tests in GitHub Actions

```yaml
- name: Install Dependencies                                  # Clean install
  run: npm ci

- name: Run Angular Unit Tests                                # Run unit tests in CI
  run: npm run test:ci -- --watch=false --browsers=ChromeHeadless  # mode

- name: Generate Coverage Report                              # Generate code coverage
  run: npm run test:ci -- --code-coverage --watch=false

- name: Upload Test Results                                   # Upload test results
  uses: actions/upload-artifact@v4                            # and coverage artifacts
  with:
    name: angular-test-results
    path: |
      junit.xml
      coverage/

- name: Publish Coverage                                      # Publish coverage to Codecov
  uses: codecov/codecov-action@v4
```

Test Results

Unit Tests	Passed	125/125
Coverage	82.4%	Threshold: 80%
Artifacts	Uploaded	junit.xml, coverage/
Build Status	Success	Passed

Best Practices

- Use headless browsers for CI (ChromeHeadless).
- Enforce minimum coverage thresholds.
- Fail the build on test failures.
- Publish test results for visibility.
- Keep tests fast, isolated, and deterministic

FULL-STACK PIPELINE

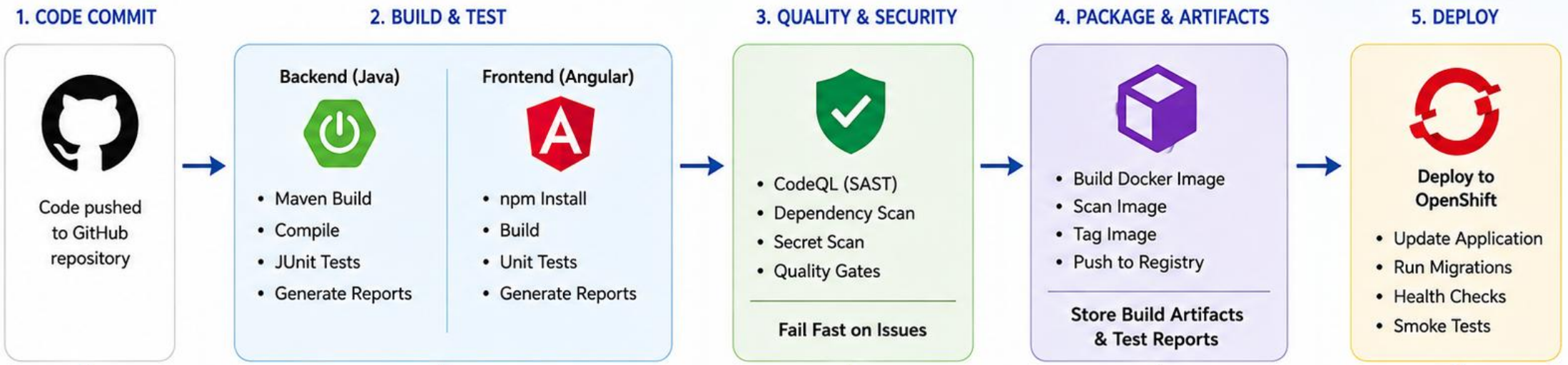
One workflow, several focused jobs, each owning exactly one concern and wired together with needs:.

Job	Responsibility
verify (backend)	Java/Maven build, tests, Surefire/Failsafe reports.
frontend-build	npm ci, Angular build and tests, dist/ artifact.
package	Package the JAR once, checksum it, and upload with the commit SHA.
deploy (manual/tag)	Consume prior artifacts only; deploy to OpenShift — never rebuild.

KEY TAKEAWAY A full-stack pipeline is several focused jobs, each owning one concern, wired together with needs: and shared artifacts — not one giant script.

Full-Stack Pipeline

An automated GitHub Actions pipeline that builds, tests, scans, and deploys the Angular + Spring Boot + PostgreSQL application to OpenShift.



BACKEND + FRONTEND BUILD STRATEGY

Treat backend and frontend as separate jobs with a shared merge gate, not one slow, coupled job.

Independent Jobs, Shared Gate

Backend and frontend jobs can run in parallel; a combined status check still blocks merge if either fails.

Path Filters Save Minutes

paths: filters can skip the Angular job on a backend-only PR, and vice versa, once the pipeline is mature.

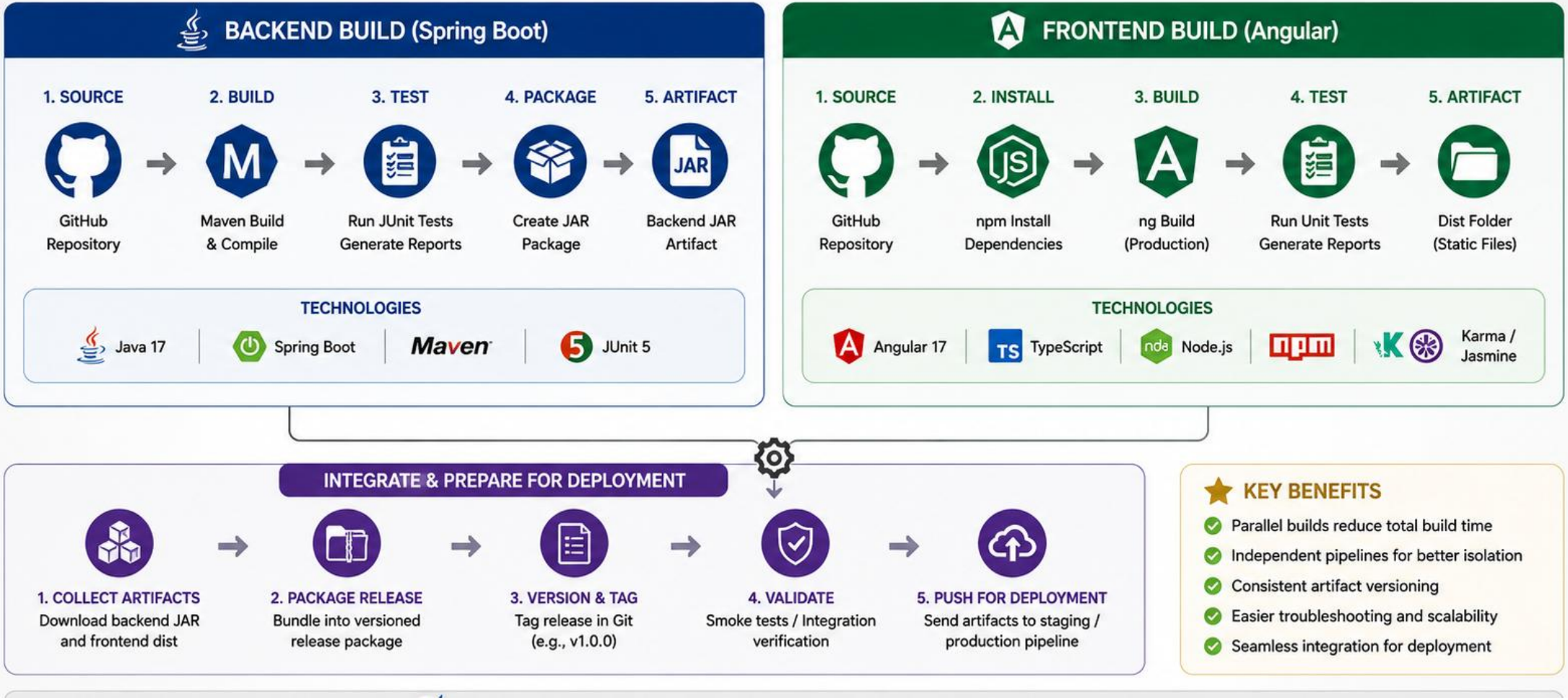
One Pipeline, Two Ecosystems

Maven and npm caches, JDK and Node runtimes, coexist in the same workflow file without conflict.

KEY TAKEAWAY Treat backend and frontend as separate jobs with a shared merge gate — don't force one slow, coupled job to do both.

BACKEND + FRONTEND BUILD STRATEGY

Build backend and frontend in parallel with isolated pipelines, then integrate into a single, versioned artifact set for deployment.



CODE SCANNING OVERVIEW

Code scanning turns 'we should look for vulnerabilities sometime' into an automatic check on every PR.

Static Analysis at Scale

Code scanning analyzes source for known vulnerability and quality patterns without executing the application.

Runs on a Schedule and on PRs

Scanning workflows commonly run on push, pull_request, and a weekly schedule to catch newly-disclosed issues too.

Results in the Security Tab

Findings appear as annotated alerts on the PR diff and in the repository's Security tab, not buried in a log file.

KEY TAKEAWAY Code scanning turns "we should look for vulnerabilities sometime" into an automatic check on every pull request.

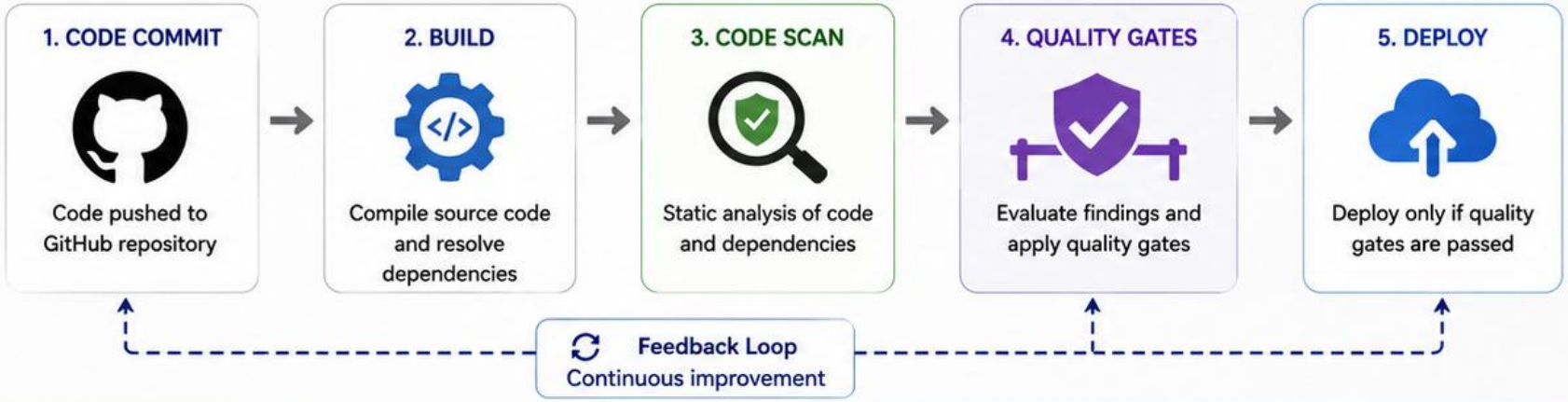
CODE SCANNING OVERVIEW

Code scanning automatically analyzes your source code and dependencies to find security vulnerabilities, code quality issues, and policy violations early in the development lifecycle.

WHY CODE SCANNING?

- Find vulnerabilities early
- Reduce risk and remediation cost
- Enforce secure coding standards
- Maintain compliance and audit readiness

HOW CODE SCANNING FITS IN THE CI/CD PIPELINE



TYPES OF SCANNING

- **SAST** (Static Application Security Testing)
  Analyzes source code for security issues
- **Dependency Scanning**
  Checks third-party libraries for known vulnerabilities
- **Secret Scanning**
  Detects hardcoded secrets, keys, and tokens
- **Container Scanning**
  Scans container images for vulnerabilities and misconfigurations

COMMON FINDINGS

- **Security Vulnerabilities**
  SQL injection, XSS, deserialization, etc.
- **Weak Configurations**
  Hardcoded credentials, insecure settings
- **Sensitive Data Exposure**
  Secrets, tokens, PII in code
- **Code Quality Issues**
  Bugs, code smells, maintainability risks

TOOLS & TECHNOLOGIES

GitHub CodeQL · SonarQube · OWASP Dependency-Check · Trivy

BENEFITS

- Shift security left and catch issues early
- Automate security and quality enforcement
- Improve code quality and developer productivity
- Support compliance and governance requirements

GITHUB CODEQL AND DEPENDENCY SECURITY SCANNING

CodeQL finds vulnerable patterns in your code; dependency scanning finds vulnerable versions in your dependencies.

GitHub CodeQL

GitHub's semantic code-analysis engine; the codeql-action workflow builds a query database and flags injection, unsafe deserialization, and more.

Dependency Security Scanning

Dependabot alerts and OWASP Dependency-Check flag known-vulnerable dependencies by CVE, with a documented severity threshold to fail the build.

KEY TAKEAWAY CodeQL finds vulnerable patterns in your code; dependency scanning finds vulnerable versions in your dependencies — a pipeline needs both.

GITHUB CODEQL

CodeQL is GitHub's semantic code analysis engine that helps you find security vulnerabilities and code quality issues by understanding your code.

CodeQL by GitHub

WHAT IS CODEQL?

CodeQL is a semantic code analysis engine that creates a database of your code and uses queries to find security vulnerabilities and code quality problems.

- understands code, not just text
- finds complex vulnerabilities
- supports many languages

HOW CODEQL WORKS IN A GITHUB ACTIONS PIPELINE

1. **INIT** — Initialize CodeQL for the language
2. **AUTOBUILD** — Build your code (or use autobuild)
3. **ANALYZE** — CodeQL analyzes code and creates a code database
4. **QUERY** — Run security and quality queries
5. **RESULTS** — Results are uploaded to GitHub Security tab

Continuous feedback

WHY USE CODEQL?

- Find security vulnerabilities early in the SDLC
- Reduce risk and prevent security incidents
- Improve code quality and maintainability
- Built-in queries + custom query support
- Seamless integration with GitHub Security

SUPPORTED LANGUAGES

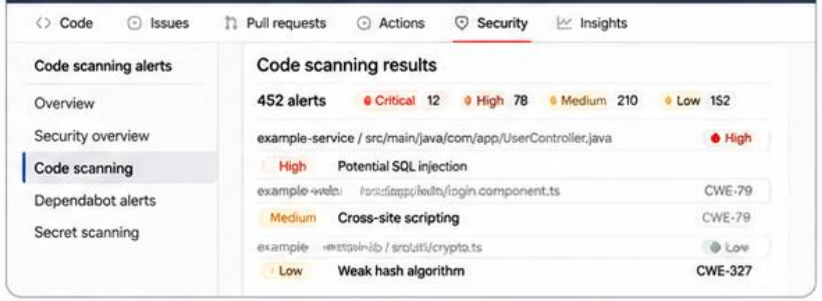
Java, C#, JavaScript, TypeScript, Python, Go ...and more

Supports 30+ languages with first-class support for Java, JavaScript, TypeScript, Python, C#, Go, and more.

TYPES OF QUERIES

Security Queries	OWASP Top 10, CWE, and other security checks
Quality Queries	Code smells, bugs, performance issues
Custom Queries	Write your own queries in CodeQL to fit your needs

RESULTS IN GITHUB



DEPENDENCY SECURITY SCANNING

Dependency scanning helps identify vulnerable components in your application's dependencies so you can fix issues early and reduce risk in your software supply chain.

Automate detection of known vulnerabilities in third-party libraries and dependencies.

WHY IT MATTERS

- Detect known vulnerabilities in open source libraries
- Reduce risk from transitive dependencies
- Fix issues early in the development lifecycle
- Improve compliance and software supply chain security

HOW DEPENDENCY SECURITY SCANNING WORKS IN CI/CD

1. **INSTALL DEPENDENCIES** — Resolve and download all project dependencies
2. **SCAN DEPENDENCIES** — Scan against vulnerability databases (CVE, OSS Index, NVD, etc.)
3. **ANALYZE RESULTS** — Identify vulnerable libraries, versions and severity levels
4. **FAIL ON RISK THRESHOLDS** — Fail build if vulnerabilities exceed configured thresholds
5. **REPORT & REMEDIATE** — Generate reports and suggest fixes or updated versions
6. **CONTINUOUS MONITORING** — Continuously rescan on new builds and dependency updates

WHAT IS SCANNED?

- **Direct Dependencies**
  Libraries you explicitly include in your project
- **Transitive Dependencies**
  Dependencies of your dependencies
- **Package Manifests**
  pom.xml, package.json, requirements.txt, go.mod, Gemfile.lock and more
- **Containers (Optional)**
  Base images and included OS packages

COMMON FINDINGS

CRITICAL	Vulnerabilities with known exploits or high impact
HIGH	Significant risk, potential for exploitation
MEDIUM	Moderate risk, limited impact or prerequisites
LOW	Low risk issues and best practice recommendations

TOOLS & INTEGRATIONS

GitHub Dependabot · OWASP Dependency-Check · Snyk · Sonatype Nexus IQ · Trivy

KEY BENEFITS

- Automated vulnerability detection for open source libraries
- Continuous monitoring of new vulnerabilities
- Actionable insights and remediation guidance
- Supports policy enforcement and compliance

BEST PRACTICES

- Keep dependencies up to date
- Pin dependency versions
- Scan on every build and pull request
- Review and fix vulnerabilities promptly
- Use allowlists and security policies

QUALITY GATES

A quality gate only works when a failing check actually turns the status red, not when it merely warns.

Definition

A quality gate is any automated check that can block a merge — tests, coverage threshold, lint, security scan.

Fail the Build, Not Just Warn

A gate only works if a failing check actually turns the status red — a warning nobody reads changes nothing.

Document the Threshold

Every gate needs a stated pass/fail line, e.g. 'no High/Critical CVEs,' recorded where the team can find it.

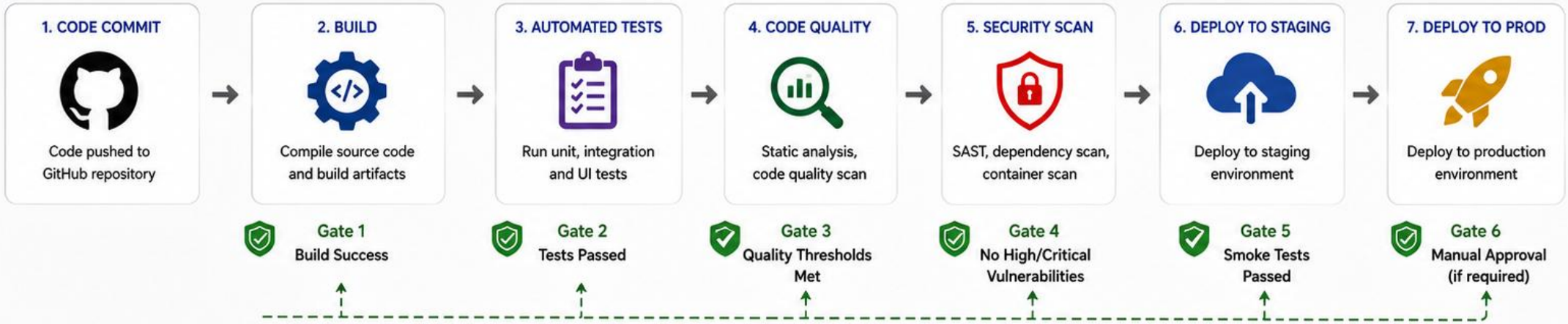
KEY TAKEAWAY A pipeline without an enforced quality gate is a build log, not a gate — enforcement is what makes it real.

QUALITY GATES

Quality gates are automated checkpoints in the CI/CD pipeline that ensure code quality, security, and reliability standards are met before moving to the next stage.

Prevent defects.
Reduce risk.
Ship with confidence.

HOW QUALITY GATES WORK IN A CI/CD PIPELINE



COMMON QUALITY GATE CHECKS

- **Build Success** — Project builds without errors or warnings (optional).
- **Code Quality** — SonarQube quality gate passed (bugs, smells, maintainability).
- **Test Coverage** — Minimum code coverage threshold is met.
- **Security** — No critical/high vulnerabilities from SAST or dependency scan.
- **Test Results** — All required unit, integration, and UI tests pass.
- **Performance / Smoke Tests** — Application responds and key endpoints are healthy.

IF A QUALITY GATE FAILS

- Pipeline stops immediately at the failed gate
- Developers are notified with clear feedback
- Issues are fixed in the code or configuration
- Pipeline is re-run to validate the fix
- Only successful builds move forward

KEY BENEFITS

- Catch defects early in the development lifecycle
- Prevent low-quality or vulnerable code from reaching production
- Ensure consistent quality and security standards
- Increase confidence in every deployment
- Support compliance and audit requirements

Quality Built-In.

BRANCH PROTECTION AND REQUIRED STATUS CHECKS

Branch protection plus required status checks turn 'tests must pass before merge' into a platform rule.

Branch Protection

Rules on main — require PRs, require reviews, block force-push — enforced by GitHub itself, not by developer discipline.

Required Status Checks

Specific workflow jobs (e.g. verify) are marked required — GitHub disables the Merge button until they're green.

KEY TAKEAWAY Branch protection plus required status checks is what makes 'tests must pass before merge' a platform rule instead of a suggestion.

BRANCH PROTECTION

Branch protection rules help enforce code quality, security, and collaboration standards by controlling how changes are made to important branches.

Protect your code.
Protect your process.
Protect your team.

WHY PROTECT BRANCHES?

- Prevent accidental or unauthorized changes
- Enforce code quality and best practices
- Reduce security risks
- Ensure stable, reliable releases

HOW BRANCH PROTECTION WORKS

1. **DEVELOPER CREATES BRANCH** — Create a feature branch from base branch
2. **OPEN PULL REQUEST** — Propose changes via Pull Request
3. **PROTECTION RULES EVALUATED** — All required checks and rules must pass
4. **REVIEW & APPROVAL** — Code review and required approvals
5. **MERGE** — Changes merged to protected branch

Feedback Loop

COMMON PROTECTION RULES

- **Require Pull Request before merging** — Disallow direct pushes to protected branches.
- **Require approvals** — Require a specific number of reviewers to approve changes.
- **Require status checks to pass** — All CI/CD and automated checks must succeed.
- **Require branches to be up to date** — Ensure the branch is up to date before merging.
- **Require signed commits** — Ensure commits are signed and verified.
- **Restrict who can push** — Limit push access to specific users or teams.
- **Require linear history** — Prevent merge commits for a clean history.
- **Delete head branches** — Automatically delete branches after merge.
- **Require conversation resolution** — All review comments must be resolved.

EXAMPLE: PROTECTION RULES FOR main BRANCH

Require pull request before merging	Enabled
Require approvals	2 reviewers required
Require status checks to pass	All checks must pass
Require branches to be up to date	Enabled
Restrict who can push	Admins only
Require signed commits	Enabled

BEST PRACTICES

- Protect main, release, and other critical branches
- Keep rules balanced (security vs. productivity)
- Use automated checks and quality gates
- Review and update rules as your team evolves
- Communicate rules clearly to the team

REQUIRED STATUS CHECKS

Required status checks ensure that all mandatory CI/CD jobs and quality gates pass before changes can be merged into a protected branch.

Quality first.
Always verified.
Always secure.

WHY REQUIRED STATUS CHECKS?

- Enforce code quality and security standards
- Prevent broken or unstable code from reaching main branches
- Increase collaboration and team accountability
- Reduce production issues and deployment risk

HOW REQUIRED STATUS CHECKS WORK

1. CODE PUSH
Developer pushes code or opens a pull request

2. CI/CD WORKFLOW
GitHub Actions workflow is triggered

3. STATUS CHECKS RUN
- Build Success
- Unit Tests Pass
- Code Quality Pass
- Security Scan Pass
- Build Artifacts Generated

4. ALL CHECKS PASS
All required checks are green

5. MERGE ALLOWED
Pull request can be merged into the protected branch

Feedback Loop
Fix issues and push again

EXAMPLE: REQUIRED STATUS CHECKS

Status Check Name	Type	Purpose	Required
build (maven)	Build	Compiles the project and validates build	✓
unit-tests	Test	Runs JUnit unit tests	✓
codeql-analysis	Code Quality	Performs static code analysis with CodeQL	✓
dependency-scan	Security	Checks dependencies for known vulnerabilities	✓
docker-build	Build	Builds Docker image	✓
e2e-tests	Test	Runs end-to-end tests	✓

WHERE TO CONFIGURE

- Code
- Pull requests
- Actions
- Projects
- Settings

Settings > Branches

Branch protection rules

Protect matching branches: main

☑ Require status checks to pass

Required status checks
- build (maven) ✓
- unit-tests ✓
- codeql-analysis ✓
- dependency-scan ✓
- docker-build ✓
- e2e-tests ✓

KEY BENEFITS

- Ensures only high-quality code is merged
- Catches issues early in the development lifecycle
- Improves reliability and stability of main branches
- Supports compliance and audit requirements
- Builds confidence in every deployment

BEST PRACTICES

- Require only essential and meaningful checks
- Keep checks fast and reliable
- Fail fast and provide clear feedback
- Review and update checks as the project evolves
- Combine with branch protection rules (reviews, linear history, etc.)

REPOSITORY VARIABLES AND GITHUB SECRETS

Three different stores for three different kinds of value — never hardcode a credential in YAML.

Store	Use For / Visibility
Repository variables (vars.)	Non-sensitive config — a region name, a feature flag. Visible in logs, safe to show.
Repository secrets (secrets.)	Sensitive values used across the whole repo — a shared API token. GitHub redacts them from logs.
Environment secrets	Sensitive values scoped to one deployment environment, e.g. test or production. Only that environment's jobs can read them.

KEY TAKEAWAY Never hardcode a credential in YAML — vars. for non-sensitive config, secrets. for sensitive values, scoped to an environment whenever a value is deploy-specific.

16 Repository Variables

Store non-sensitive configuration values used across workflows.

What are Repository Variables?

Repository variables allow you to store configuration values that are not sensitive, such as environment names, feature flags, or paths. These values can be reused across multiple workflows without hardcoding.

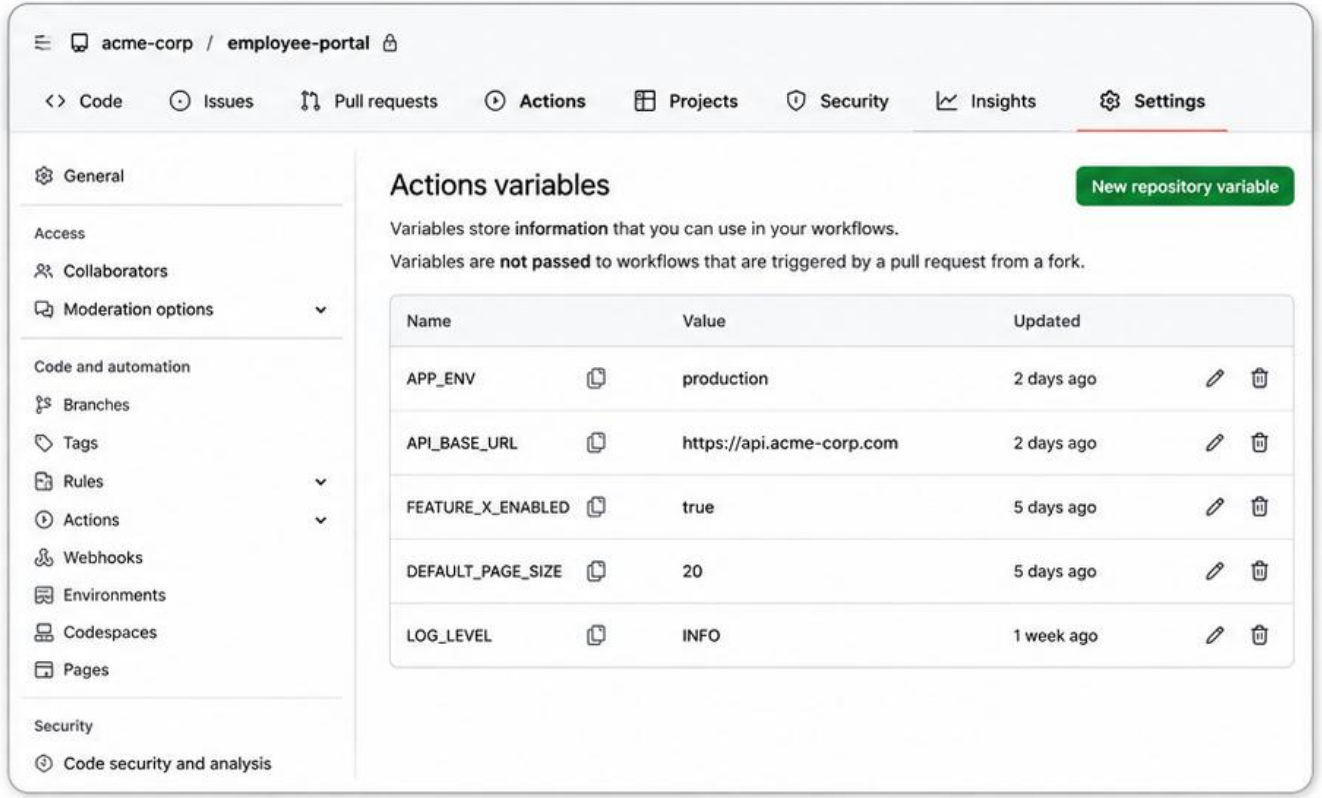
Key Benefits

- Centralized configuration management
- Reusable across jobs and workflows
- Versioned with the repository settings

How to Use

Access variables in workflows using the `vars` context.

```
env:
  APP_ENV: ${{ vars.APP_ENV }}
  LOG_LEVEL: ${{ vars.LOG_LEVEL }}
```



Best Practice: Use repository variables for values that may change between environments but are not sensitive, such as URLs, feature flags, or default settings.

44 GitHub Secrets

Store sensitive information securely and use it in GitHub Actions.

What are GitHub Secrets?

GitHub Secrets allow you to store sensitive information such as passwords, tokens, API keys, and private certificates.
Secrets are encrypted and not exposed in logs or workflow output.

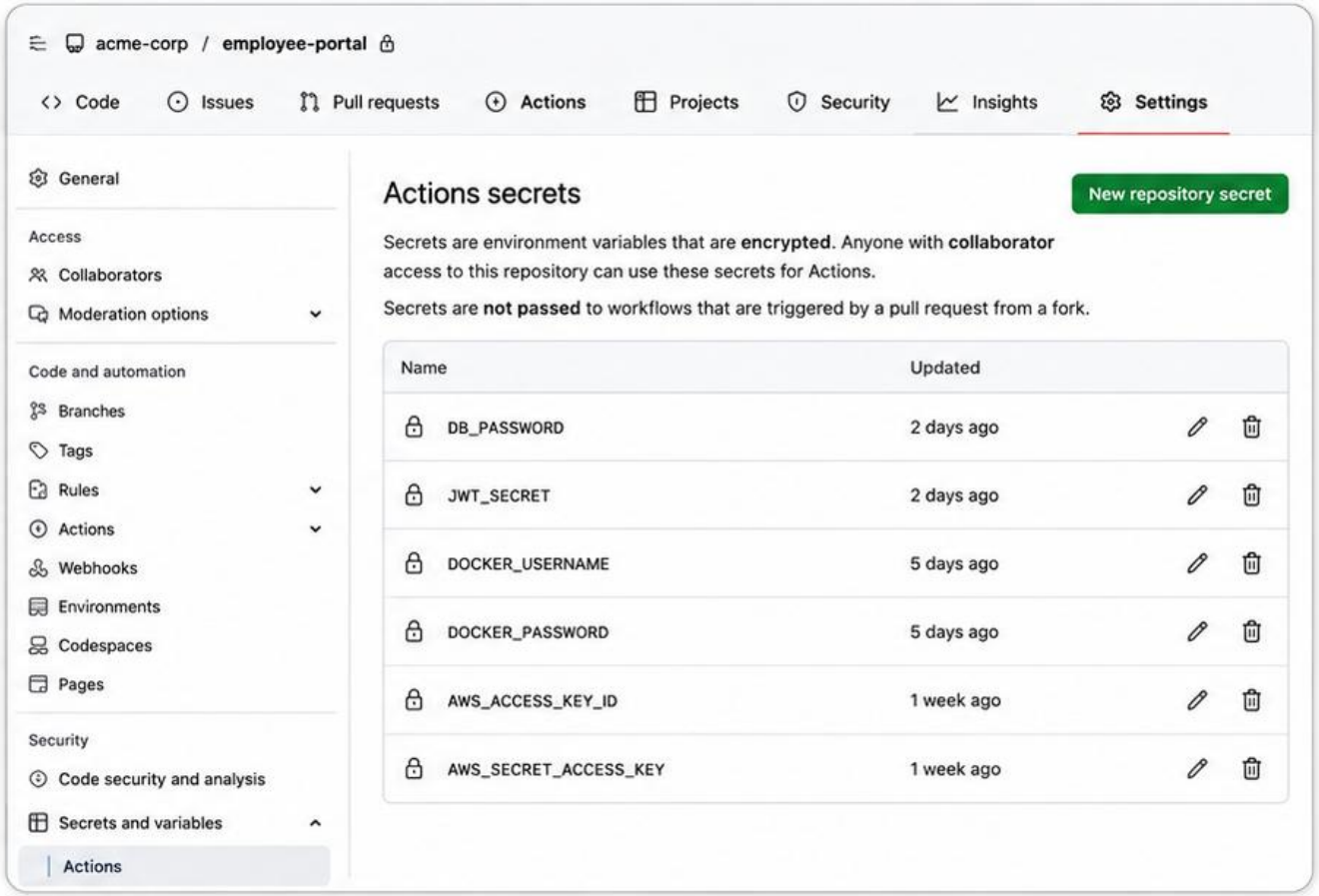
Key Benefits

- Secure storage of sensitive data
- Encrypted at rest and in transit
- Automatically masked in logs
- Fine-grained access (repo / environment / org)

How to Use

Access secrets in workflows using the `secrets` context.

```
steps:
 - name: Login to Docker Hub
   run: echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u
        ${{ secrets.DOCKER_USERNAME }} --password-stdin
```



Best Practice: Use secrets for sensitive data only. Rotate secrets regularly and prefer OIDC or short-lived tokens when possible.

45 Environment Secrets

Store sensitive information scoped to a specific environment.

What are Environment Secrets?

Environment secrets are encrypted secrets that are available only to workflows that reference a specific environment. They add an extra layer of protection for sensitive data used in deployments.

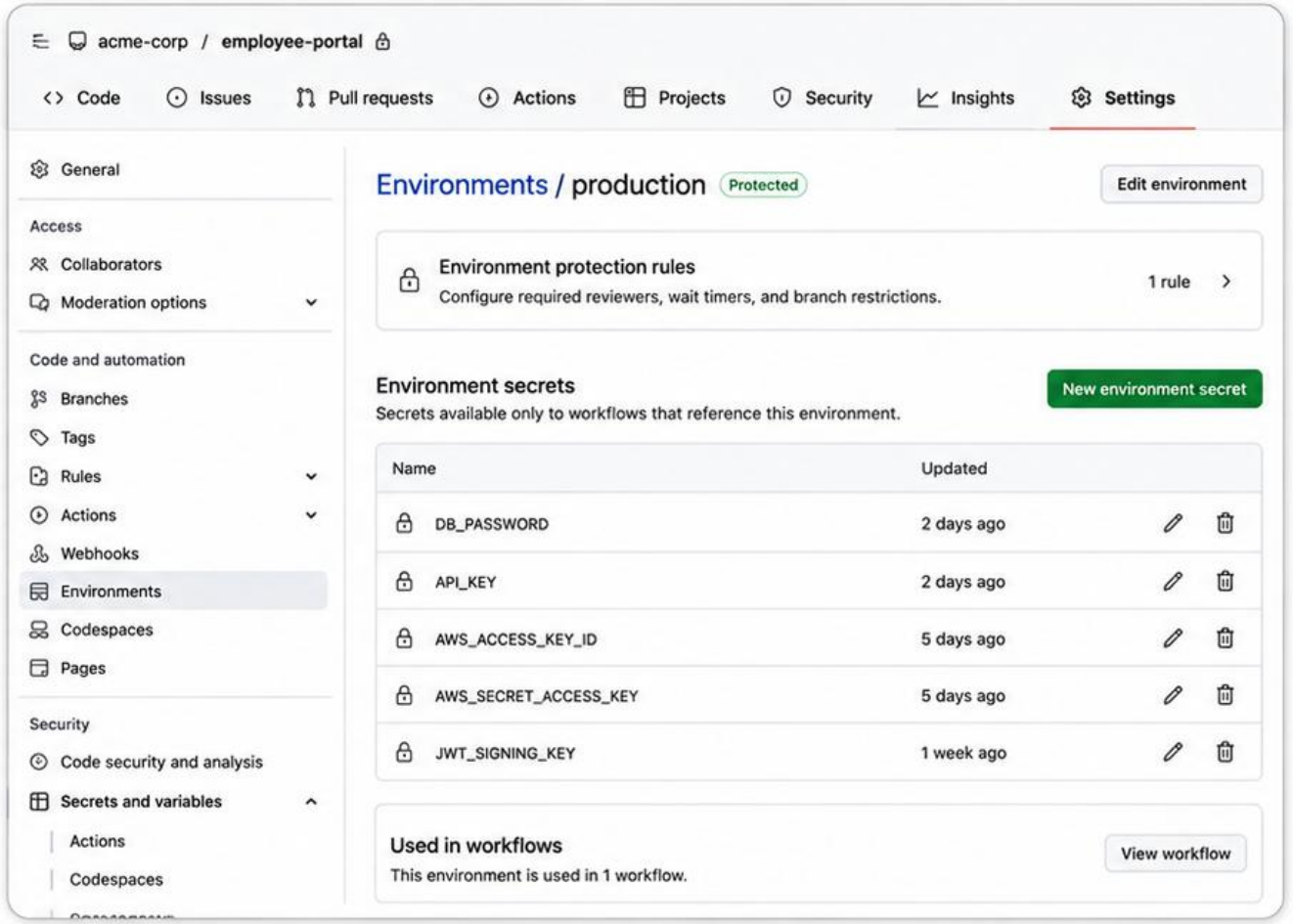
Key Benefits

- Scoped to a specific environment (e.g., staging, production)
- Not accessible to other environments
- Supports protection rules and approvals
- Works with environment-level variables and OIDC

How to Use

Reference environment secrets in workflows using the `secrets` context.

```yaml
jobs:
  deploy:
    environment: production
    steps:
    - name: Deploy to Production
      run: ./deploy.sh
      env:
        DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
```



Best Practice: Use environment secrets for production credentials and sensitive keys. Enable protection rules (reviewers, wait timers) for production environments.

TRY IT YOURSELF — EXERCISE 5: ACTIONS SECRETS CHECKLIST

Reinforces: sorting which names live in Actions secrets vs plain vars, and never echoing values.

STEPS (notes/lab43-secrets-checklist.md)

Step	What To Do
1 -- Sort	Sort: workflow YAML, README, registry password, kubeconfig, .env, scan reports.
2 -- Check the Reference	Only non-secret config in Git; credentials in Actions secrets/variables as instructed.
3 -- Leak Response	Write three steps if a secret is committed: rotate, purge history per policy, notify instructor.
4 -- CRM Note	Customer fixtures are not secrets -- but real customer dumps are forbidden.

Secrets (names only)

```
CRM_REGISTRY_USER   (secured, deployment=test)
CRM_REGISTRY_TOKEN  (secured, deployment=test)
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-secrets-checklist.md (workspace: examples/module-43-exercises).

TRY IT YOURSELF — EXERCISE 6: OUTLINE CI RUNBOOK

Reinforces: outlining how a peer re-runs failed verify, finds artifacts, and locates secrets by name.

STEPS (notes/lab43-ci-runbook-outline.md)

Step	What To Do
1 -- Headings	Triggers, jobs, where reports live, how to re-run, what deploy steps exist (none yet).
2 -- Re-run Recipe	Bullet the GitHub UI/CLI re-run path and the local ./mvnw -B clean verify equivalent.
3 -- Evidence Index	Placeholder links for the Surefire zip and the JAR SHA artifact names.
4 -- Scope	Mark as a pre-lab outline for Lab 43 -- the full runbook is written during the lab.

docs/ci-runbook.md skeleton

```
## Triggers  ## Jobs  ## Re-run
## Evidence  ## Secrets (names only)
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-ci-runbook-outline.md (workspace: examples/module-43-exercises).

OIDC OVERVIEW

OIDC replaces a stored, long-lived credential with a short-lived, signed token requested fresh per run.

What OIDC Replaces

OpenID Connect lets a workflow request a short-lived, cryptographically-signed token instead of storing a long-lived cloud credential as a secret.

Trust, Not Secrets

The cloud provider (or OpenShift) is configured to trust GitHub's OIDC issuer for a specific repo/branch — no password ever leaves GitHub.

Why It's Safer

A leaked OIDC token expires in minutes and is scoped to one run; a leaked long-lived secret is valid until someone rotates it.

KEY TAKEAWAY OIDC is the modern alternative to storing deploy credentials as secrets at all — prefer it wherever the target platform supports it.

46 OIDC Overview

Use OpenID Connect to securely authenticate GitHub Actions workflows to cloud providers without long-lived secrets.

What is OIDC?

OpenID Connect (OIDC) is an identity layer built on OAuth 2.0. It allows GitHub Actions to obtain short-lived identity tokens to authenticate with cloud providers securely.

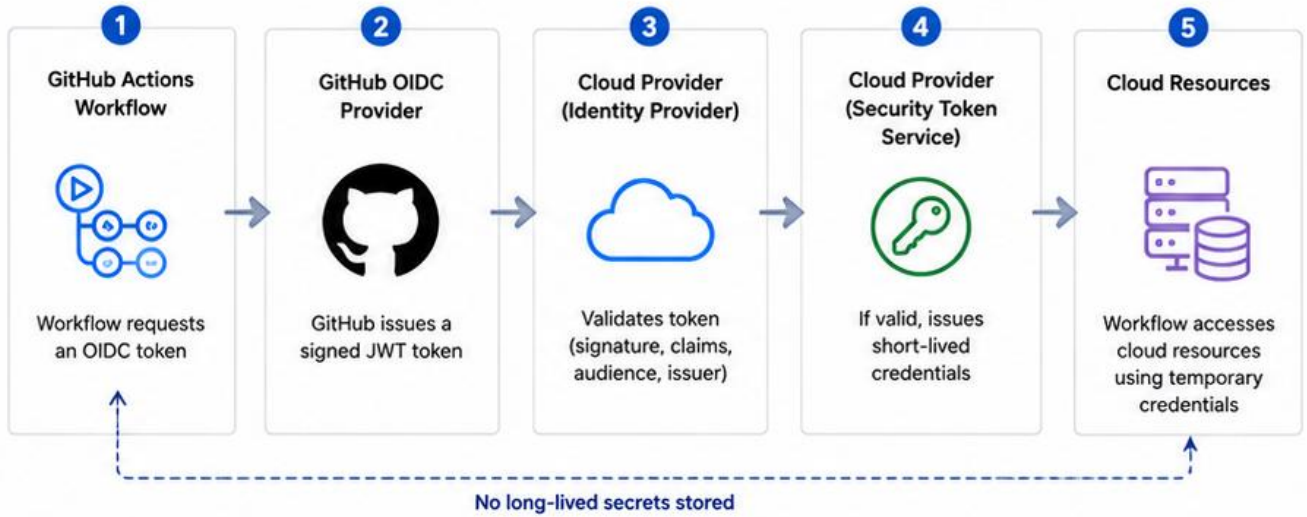
Key Benefits

- **No long-lived credentials:** Eliminates static cloud keys and secrets
- **Short-lived tokens:** Tokens are time-bound and automatically rotated
- **Fine-grained access:** Control access using claims and conditions
- **Audit-friendly:** Cloud provider logs show federated identity usage
- **Zero trust:** Workflows are trusted without storing credentials

How It Works

1. GitHub Actions requests an OIDC token from GitHub.
2. GitHub issues a signed JWT (JSON Web Token) to the workflow.
3. The cloud provider validates the token using GitHub's identity provider.
4. If valid, the provider issues temporary credentials to the workflow.
5. The workflow uses those credentials to access cloud resources.

OIDC Authentication Flow



Common OIDC Claims Used

Claim	Description
iss	Issuer of the token (https://token.actions.githubusercontent.com)
sub	Subject (e.g., repo:owner/repo:ref:refs/heads/main)
aud	Audience (typically the cloud provider)
exp	Expiration time
nbf	Not before time
jti	JWT ID (unique identifier)

Where OIDC Is Used

- AWS (IAM Roles for OIDC)
- Azure (Workload Identity Federation)
- Google Cloud (Workload Identity Pool)
- Vault / Other OIDC-compatible providers

Best Practice: Use OIDC with least-privilege roles and claim conditions (e.g., branch, environment) to restrict access.

DEPLOYMENT ENVIRONMENTS AND MANUAL APPROVALS

Environments turn 'only certain people can deploy to production' into an enforced GitHub setting.

Deployment Environments

GitHub Environments (test, staging, production) each carry their own secrets, variables, and protection rules.

Manual Approvals

An environment can require a named reviewer to approve before a job deploying to it is allowed to run.

KEY TAKEAWAY Environments turn 'only certain people can deploy to production' into an enforced GitHub setting, not a verbal policy.

47 Deployment Environments

Organize, protect, and control where your applications are deployed.

What are Environments?

GitHub Environments let you manage deployments to different stages (e.g., dev, staging, production). They provide visibility, protection rules, and secrets specific to each environment.

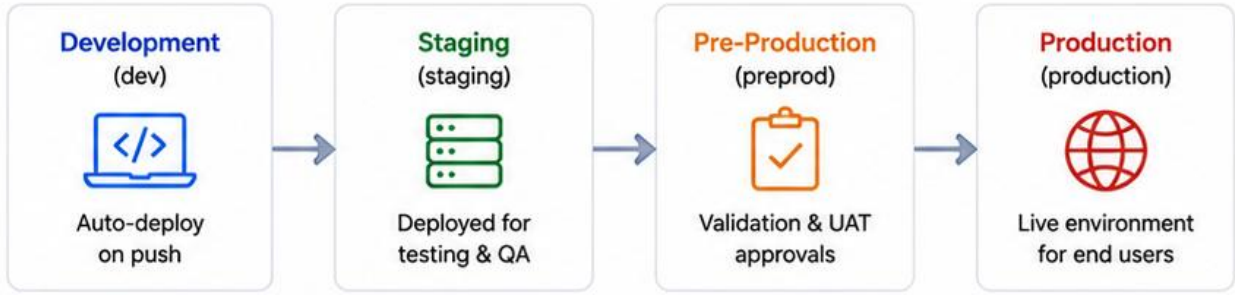
Key Benefits

- Organize deployments across stages
- Apply protection rules and approvals
- Use environment-scoped secrets
- Track deployment history and status
- Prevent accidental production deployments

How It Works

1. Define environments in your repository settings.
2. Configure protection rules (e.g., required reviewers, wait timers).
3. Use environments in your workflow jobs.
4. Deployments are tracked and require approval (if configured).

Typical Deployment Pipeline



Environment Protection Rules

Required Reviewers
Require one or more reviewers to approve deployments.

Wait Timers
Add a wait period before deployment is executed.

Deployment Branches
Limit deployments to specific branches or tags.

Environment Secrets
Store secrets scoped only to this environment.

Example Workflow Usage

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production    # Targets the "production" environment
    steps:
      - name: Deploy to Production
        run: ./deploy.sh
```

Deployment Visibility

- View deployment history
- See who approved and when
- Track status (in progress, success, failure)
- Rollback if needed

Best Practice: Use environments to enforce approval gates and protect production. Keep environment configurations consistent and auditable.

48 Manual Approvals

Require human review and approval before running important jobs.

What Are Manual Approvals?

Manual approvals add a human gate in your workflow before proceeding to sensitive actions such as production deployments. They improve safety, compliance, and change control.

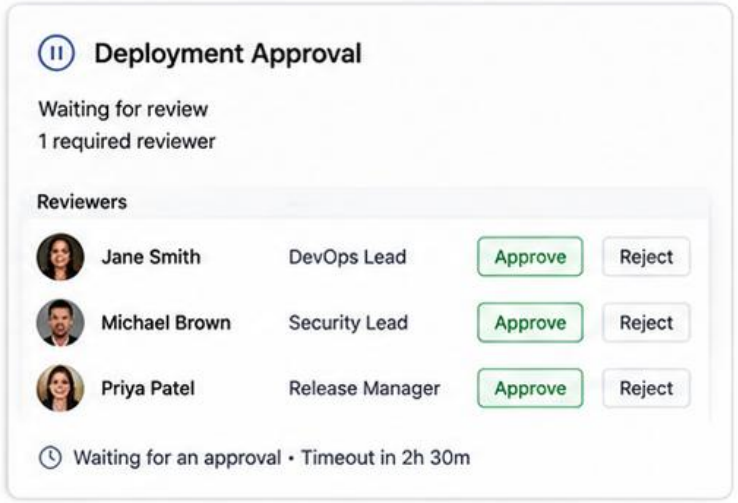
Key Benefits

- Prevent unintended deployments
- Ensure human review before critical steps
- Meet compliance and audit requirements
- Support environment protection rules
- Provide clear approval history

How It Works

1. Add a job that requires manual approval.
2. Workflow pauses and waits for approved reviewers.
3. An approver reviews the run and approves or rejects.
4. If approved, the workflow continues to the next job.
5. If rejected or timed out, the workflow stops.

Example Deployment Workflow with Manual Approval



Where Manual Approvals Are Used

- Production deployments
- Changes to critical environments
- Infrastructure changes
- Security-sensitive actions
- Compliance and audit requirements

Tip: Combine manual approvals with environment protection rules (required reviewers, wait timers).

Best Practice: Use manual approvals for production and high-impact changes. Keep the list of approvers small and accountable.

REUSABLE WORKFLOWS AND COMPOSITE ACTIONS

Reusable workflows share a whole job across repositories; composite actions share a handful of steps within one.

REUSABLE WORKFLOWS

- workflow_call lets one workflow invoke another by reference — share a verify job across many repos.
- Inputs and secrets are passed explicitly, keeping the contract visible.

COMPOSITE ACTIONS

- A composite action bundles several steps into one reusable uses: step, defined in an action.yml.
- Best for repeating the same 3-4 steps (checkout + setup-java + cache) across many jobs in one repo.

Calling a reusable workflow

```
jobs:
  verify:
    uses: org/shared-workflows/.github/workflows/java-verify.yml@v1
    with: { java-version: "21" }
```

KEY TAKEAWAY Reusable workflows share a whole job across repositories; composite actions share a handful of steps within one — pick the smallest tool that removes the duplication.

49 Reusable Workflows

Share and reuse workflows across repositories and organizations.

What are Reusable Workflows?

Reusable workflows allow you to define a workflow once and call it from multiple repositories or workflows. They promote consistency, reduce duplication, and simplify maintenance.

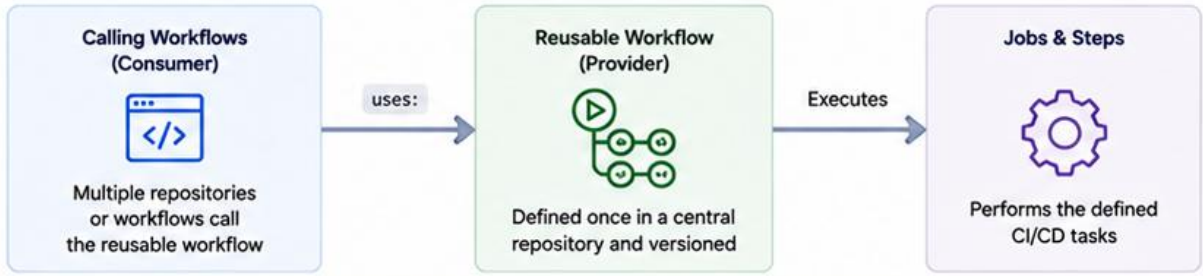
Key Benefits

- Reuse common CI/CD logic across repositories
- Enforce standard processes and compliance
- Centralize updates in one place
- Reduce duplication and maintenance effort
- Improve reliability and consistency

How It Works

1. Create a reusable workflow in a central repository.
2. Expose inputs, secrets, and outputs.
3. Call the reusable workflow using the `'uses:'` keyword.
4. Pass inputs and secrets as needed.
5. The called workflow executes and returns outputs.

Reusable Workflow Architecture



1. Define Reusable Workflow (Provider Repository)

.github/workflows/build.yml

```yaml
on:
  workflow_call:
    inputs:
      java-version:
        required: true
        type: string
      run-tests:
        required: false
        type: boolean
        default: true
    secrets:
      maven-settings-token:
        required: true
    outputs:
      build-artifact:
        description: 'Build artifact name'
        value: ${{ jobs.build.outputs.artifact }}

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      artifact: ${{ steps.set-output.outputs.name }}
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: ${{ inputs.java-version }}
      - name: Build with Maven
        run: mvn -B package
      - name: Set output
        id: set-output
        run: echo "name=app.jar" >> $GITHUB_OUTPUT
```

2. Call Reusable Workflow (Consumer Repository)

.github/workflows/ci.yml

```yaml
name: CI Pipeline

on:
  push:
    branches: [ main ]

jobs:
  ci:
    uses: org-name/central-workflows/.github/workflows/build.yml@v1
    with:
      java-version: '17'
      run-tests: true
    secrets:
      maven-settings-token: ${{ secrets.MAVEN_TOKEN }}

  deploy:
    needs: ci
    runs-on: ubuntu-latest
    steps:
      - name: Download artifact
        uses: actions/download-artifact@v4
        with:
          name: ${{ needs.ci.outputs.build-artifact }}

      - name: Deploy
        run: ./deploy.sh
```

Key Capabilities

Cross-Repository
Reuse workflows across multiple repositories.

Secure
Pass secrets securely to reusable workflows.

Flexible Inputs
Accept inputs to customize behavior.

Outputs
Return outputs to calling workflows.

Versioned
Use versions (tags/branches) for stability and control.

Best Practice: Create a dedicated repository for reusable workflows, version them with tags, and keep interfaces stable. Document inputs, secrets, and outputs for easy adoption.

50 Composite Actions

Create your own custom actions by combining multiple steps.

What are Composite Actions?

Composite actions allow you to create reusable actions using a sequence of workflow steps.
They run directly on the runner, do not create separate jobs, and are version-controlled in your repository.

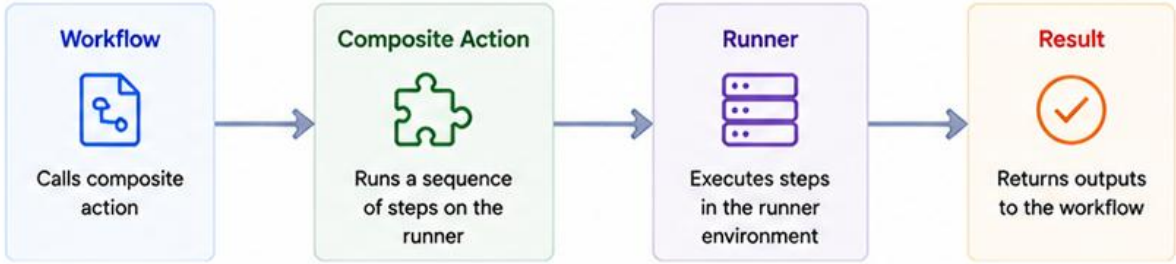
Key Benefits

- Encapsulate and reuse common step sequences
- Share actions across workflows and repositories
- Version and maintain in source control
- Easier to update and improve over time
- No Docker build required (runs on the runner)

How It Works

1. Create a directory with an action.yml file.
2. Define inputs, outputs, and the sequence of steps.
3. Commit and version the composite action.
4. Reference the action in your workflows using `uses:`.
5. The composite action runs the defined steps on the runner.

How Composite Actions Work



Example: action.yml (Composite Action Definition)

.github/actions/setup-java/action.yml

```yaml
name: 'Setup Java Environment'
description: 'Setup Java with Maven cache'
inputs:
  java-version:
    description: 'Java version to use'
    required: true
    defualt: '17'
outputs:
  java-version:
    description: 'The Java version used'
    value: ${{ inputs.java-version }}
runs:
  using: 'composite'
  steps:
    - name: Setup JDK
      uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: ${{ inputs.java-version }}
        cache: 'maven'

    - name: Print Java Version'
      shell: bash
      run: java -version
```

Example: Using Composite Action in a Workflow

.github/workflows/ci.yml

```yaml
name: CI Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Java (Composite Action)
        uses: org-name/ci-actions/setup-java@v1
        with:
          java-version: '17'

      - name: Build with Maven
        run: mvn -B package

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: app
          path: target/*.jar
```

Key Capabilities

Inputs
Accept parameters to customize behavior.

Outputs
Return values to the calling workflow.

Steps
Run any combination of actions or shell commands.

Versioned
Tag releases and use semantic versions.

No Docker Required
Runs steps directly on the GitHub-hosted runner.

Best Practice: Use composite actions to standardize common tasks (e.g., setup, caching, publishing) and keep workflows clean and maintainable.

CONTAINER BUILD IN GITHUB ACTIONS

Containerizing the build is the same package-once discipline as the JAR-plus-checksum pattern.

docker/build-push-action

The standard action for building an image from a Dockerfile inside a workflow step, with layer caching support.

Build Once, Tag by Commit

Tag the image with the Git SHA (and a semantic version on release) so every image is traceable to exact source.

Multi-Stage Dockerfiles

A build stage compiles the JAR; a slim runtime stage copies only the JAR — keeping the shipped image small.

KEY TAKEAWAY Containerizing the build is the same package-once discipline as the JAR-plus-checksum pattern — build the image once, then only ever promote it.

MODULE 43 GitHub Actions and CI/CD Integration

GitHub Actions

Container Build in GitHub Actions

Build a **Docker container image** from your application and push it to a **container registry** as part of your CI/CD pipeline.

KEY BENEFITS

Consistent Builds
Reproducible, environment-independent container images.

Portable
Run anywhere – dev, test, staging, or production.

Faster Delivery
Automate build and push to speed up deployment.

Registry Ready
Push images to registries like GHCR, Docker Hub, or private registries.

.github/workflows/build-container.yml

```yaml
name: Build and Push Container Image
on:
  push:
    branches: [ main ]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: ghcr.io/${{ github.repository }}:latest
```

GitHub Container Registry (ghcr.io)

Ready for Deployment

CONTAINER BUILD FLOW

1. **Checkout** – Get the source code from the repository.
2. **Build Image** – Build the Docker image using the Dockerfile.
3. **Tag Image** – Tag the image with commit SHA, version or latest.
4. **Push Image** – Push the tagged image to the container registry.
5. **Image Ready** – Image is available for deployment pipelines.

BEST PRACTICES

- ✓ Use multi-stage builds to keep images small.
- ✓ Tag images with meaningful versions.
- ✓ Scan images for vulnerabilities.
- ✓ Push to a secure private registry.
- ✓ Automate builds on every commit or PR.

RESULT

A secure, versioned container image built and pushed automatically, ready for deployment.

PUBLISHING DOCKER IMAGES

Publishing is just another job that pushes a tagged, traceable image — it never rebuilds from source.

WHERE IMAGES GO

- GitHub Container Registry (ghcr.io) is the default choice for a repo already on GitHub.
- An internal OpenShift-integrated registry is common in enterprise clusters — login uses OIDC or a scoped secret.

AUTHENTICATION

- docker/login-action logs in with a token, never a hardcoded password in the workflow file.
- GITHUB_TOKEN has enough scope to push to ghcr.io for the same repository automatically.

Publish to GHCR

```
- uses: docker/login-action@v3
  with: { registry: ghcr.io, username: ${{ github.actor }},
          password: ${{ secrets.GITHUB_TOKEN }} }
- uses: docker/build-push-action@v6
  with: { push: true, tags: ghcr.io/org/crm:${{ github.sha }} }
```

KEY TAKEAWAY Publishing is just another job that consumes the build's artifacts and pushes a tagged, traceable image — it never rebuilds from source.

GitHub Actions

PUBLISHING DOCKER IMAGES

Push your Docker images to a container registry so they can be shared, pulled, and deployed anywhere.

KEY BENEFITS

Centralized Storage
Store and manage images in a secure container registry.

Share and Reuse
Share images across teams and environments.

Reliable Deployments
Pull the exact version of the image for consistent deployments.

Versioned and Traceable
Use tags and digests to track and roll back images easily.

.github/workflows/docker-publish.yml

```yaml
- name: Push Docker image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ghcr.io/${{ github.repository }}:${{ github.ref_name }}
    username: ${{ github.actor }}
```

PUBLISHING WORKFLOW

1. **Build Image** — Build the Docker image from your application.
2. **Tag Image** — Tag the image with the registry URL and version.
3. **Authenticate** — Login to the container registry securely.
4. **Push Image** — Push the tagged image to the registry.
5. **Published** — Image is available for pull and deployment.

CONTAINER REGISTRIES

- docker hub
- GitHub Container Registry (GHCR)
- Amazon Elastic Container Registry (ECR)
- Google Artifact Registry (GAR)
- Azure Container Registry (ACR)

BEST PRACTICES

- ✓ Use meaningful and consistent tag names.
- ✓ Push immutable versions (avoid using latest in production).
- ✓ Authenticate using securely stored secrets.
- ✓ Scan images for vulnerabilities before pushing.
- ✓ Automate image cleanup with lifecycle policies

TYPICAL IMAGE TAGGING STRATEGY

v1.2.3	Semantic version for releases
1.2.3-build.45	Build-specific version
latest	Latest stable (use with caution)

DEPLOYMENT TO OPENSHIFT

Deploying to OpenShift is the last link in the chain — promote the exact artifact already verified.

oc CLI in a Workflow Step

A deploy job installs or uses the oc CLI to log in (oc login --token=...) and apply or roll out a new image tag.

Never Rebuild in Deploy

The deploy job consumes the image/JAR the package job already produced — rebuilding here breaks the chain of custody.

Gated by Environment

The OpenShift deploy job targets a GitHub Environment (staging, production) with its own secrets and required approvals.

KEY TAKEAWAY Deploying to OpenShift is the last link in the chain — verify, package once, then promote that exact artifact, never a fresh rebuild, onto the cluster.

GitHub Actions

DEPLOYMENT TO OPENSHIFT

Deploy your containerized application to OpenShift using automated, secure, and repeatable pipelines.

KEY BENEFITS

Enterprise-Ready Platform
Run at scale with security, resiliency, and high availability.

Automated Deployments
Consistent, repeatable deployments with GitOps and CI/CD.

Faster Releases
Reduce manual steps and accelerate time to production.

Observability & Control
Built-in monitoring, logging, and rollout strategies.

DEPLOYMENT PIPELINE OVERVIEW

1. **Build Image** — Build and push Docker image to registry.
2. **Push to Registry** — Image stored in container registry.
3. **Update Manifests** — Update OpenShift manifests (YAML).
4. **Commit Changes** — Push manifest changes to GitHub.
5. **GitHub Actions** — Pipeline triggers deployment to OpenShift.
6. **Deploy to OpenShift** — Application deployed and available.

OPENSHIFT DEPLOYMENT FLOW

Developer — Push code & manifests to GitHub
→ **GitHub Repository** — Source code + OpenShift manifests
→ **GitHub Actions** — Build, Test, Scan & Deploy
→ **OpenShift Cluster** — Deploys application automatically
→ **Live Application** — Accesible via route / domain

.github/workflows/deploy-openshift.yml

```yaml
- name: Deploy to OpenShift
  uses: redhat-actions/oc-new-app@v1
  with:
    openshift_server_url: ${{ secrets.OPENSHIFT_SERVER }}
    openshift_token: ${{   secrets.OPENSHIFT_TOKEN }}
    namespace: my-app
    image: ghcr.io/${{ github.repository }}:${{ github.sha }}
    resources: manifests/
    reuse_existing: true
```

BEST PRACTICES

- ✓ Use namespaces to isolate environments (dev, test, prod).
- ✓ Store OpenShift credentials in GitHub Secrets.
- ✓ Use rollout strategies: Rolling, Blue/Green, or Canary.
- ✓ Enable readiness & liveness probes for applications.
- ✓ Monitor deployments and set alerts for failures.

DEPLOYMENT STRATEGIES

Rolling Update
Gradual replacement of pods with zero downtime.

Blue/Green Deployment
Switch traffic between two identical environments.

Canary Deployment
Deploy to a small subset of users first.

OBSERVABILITY IN OPENSHIFT

Logs
Centralized logging with Elasticsearch + Kibana

Metrics
Monitor with Prometheus and Grafana

Alerts
Configure alerting rules and notifications

Events
Track resource and deployment events in real time.

CI/CD FAILURE DIAGNOSIS

Most pipeline failures trace back to one of a small, learnable set of causes — check these first.

Symptom	Likely Cause → Fix
Pipeline can't find pom.xml	Wrong working directory / monorepo → cd to the module, or set working-directory:.
Tests pass locally, fail in CI	JDK/Maven/Node version drift → pin exact versions with setup-java/setup-node.
Cache never hits	Cache key/path mismatch → use the default cache: maven / cache: npm behavior.
Empty or missing artifact	Wrong path: glob, or the build step never ran → match the glob to the real target/ or dist/ output.
Secret appears in a log	echo of a secret variable, or set -x tracing → remove the echo; rotate the secret; never trace over secrets.

KEY TAKEAWAY Most pipeline failures trace to one of five causes — wrong directory, version drift, cache mismatch, wrong artifact path, or a leaked secret — check those first.

GitHub Actions

CI/CD FAILURE DIAGNOSIS

Identify, analyze, and resolve pipeline failures quickly to keep your delivery process reliable and efficient.

COMMON FAILURE AREAS

Code & Build
Compilation errors, test failures, dependency issues.

Tests
Unit, integration, or UI tests failing or timing out.

Security Scans
Vulnerabilities, policy violations, or quality gate failures.

Artifacts
Build or upload/download artifact failures.

Deployment
Environment misconfiguration, permission, or rollout errors.

QUICK TIPS

- Reproduce locally when possible.
- Use smaller, isolated reruns.
- Add more logging and assertions.
- Automate notifications for failures.

FAILURE DIAGNOSIS PROCESS

1. **Detect** — Identify failed job or step.
2. **Investigate** — Review logs, error messages, and stack traces.
3. **Analyze** — Determine root cause using logs, metrics, and context.
4. **Fix** — Update code, configuration, or pipeline steps.
5. **Validate** — Re-run pipeline or affected stage to confirm fix.
6. **Prevent** — Add checks, alerts, and documentation to prevent recurrence.

TROUBLESHOOTING CHECKLIST

- Check the failed job and step.
- Read the full logs (expand collapsed sections).
- Look for the first error, not the last.
- Verify environment variables and secrets.
- Validate dependencies and versions.
- Confirm service availability (DB, APIs, registries).
- Check permissions and access tokens.
- Review recent changes/commits.
- Re-run with debug logging if needed.
- Document the issue and resolution.

LOG EXAMPLE (TRUNCATED)

```
2025-05-19T10:15:23Z  [build] Starting step: Run Maven Tests
2025-05-19T10:16:02Z  [INFO]  -------------------------------------------------------
2025-05-19T10:16:02Z  [ERROR]  Tests run: 45, Failures: 2, Errors: 0, Skipped: 1
2025-05-19T10:16:02Z  [ERROR]  Failures:
2025-05-19T10:16:02Z  [ERROR]    UserServiceTest.testCreateUser: expected true but was false
2025-05-19T10:16:02Z  [INFO]  -------------------------------------------------------
2025-05-19T10:16:02Z  ##[error]Process completed with exit code 1.
```

USEFUL DIAGNOSTIC TOOLS

GitHub Actions Logs
Detailed logs for each step and job.

Artifacts & Reports
Download reports, test results, and screenshots.

Step Summary
Review annotations and failure summaries.

Actions Insights
Analyze workflow runs, trends, and durations.

Re-run Jobs
Re-run failed jobs or steps without new commit.

BEST PRACTICES

- Fail fast and clearly.
- Keep steps small and focused.
- Use meaningful names for jobs/steps.
- Monitor flaky tests and reduce them.
- Continuously improve pipelines.

ENTERPRISE PIPELINE BEST PRACTICES

An enterprise-grade pipeline is boring on purpose — pinned, gated, least-privilege, and documented.

Pin Everything

Actions, JDK, Node, and base images are all pinned to explicit versions — 'latest' is a silent, unreviewed change waiting to happen.

Fail Loud, Fail Fast

A red status check plus a clear job/step name beats a green build that quietly skipped something.

Least Privilege

GITHUB_TOKEN and any deploy credential get the minimum scope and shortest lifetime the job actually needs.

Documented and Reviewable

The workflow YAML and a short runbook are reviewed like code — no pipeline change ships without eyes on it.

KEY TAKEAWAY An enterprise-grade pipeline is boring on purpose — pinned versions, enforced gates, least-privilege secrets, and a runbook a peer can follow without you.

GitHub Actions

ENTERPRISE PIPELINE BEST PRACTICES

Design secure, reliable, maintainable, and fast CI/CD pipelines that scale with your organization.

1. AUTOMATE EVERYTHING
- Automate build, test, scan, and deploy.
- Eliminate manual steps to reduce errors.

2. FAIL FAST
- Catch issues early in the pipeline.
- Shift left with tests and quality checks.

3. SMALL & FREQUENT COMMITS
- Encourage small, focused changes.
- Enables faster feedback and easier rollbacks.

4. QUALITY GATES
- Enforce code quality, test coverage, and security thresholds.
- Block bad code from reaching production.

5. SECURE BY DESIGN
- Scan code, dependencies, containers, and IaC.
- Manage secrets securely using GitHub Secrets and OIDC.

6. INFRASTRUCTURE AS CODE
- Use Terraform/Ansible for repeatable infrastructure.
- Version and review infrastructure changes.

7. BRANCH STRATEGY
- Use trunk-based development.
- Protect main branches with required checks.

8. OBSERVABILITY
- Monitor pipeline and application health.
- Centralize logs, metrics, and alerts.

9. REUSABILITY
- Use reusable workflows and composite actions.
- DRY pipelines for consistency.

10. ENVIRONMENT PARITY
- Keep dev, test, staging, and prod as similar as possible.
- Use containers and IaC.

11. PROGRESSIVE DELIVERY
- Use rolling updates, blue/green, or canary deployments.
- Minimize risk to users.

12. DOCUMENTATION
- Document pipeline design, steps, and runbooks.
- Keep documentation up to date.

13. CONTINUOUS IMPROVEMENT
- Measure pipeline performance.
- Optimize for speed, reliability, and cost.

14. COST OPTIMIZATION
- Use self-hosted runners or right-sized runners.
- Cache dependencies and build outputs.

IDEAL ENTERPRISE PIPELINE STAGES

1. **CODE** — Commit & push code to repository
2. **BUILD** — Compile, lint, and build artifacts
3. **TEST** — Run unit, integration, and UI tests
4. **SECURITY** — SAST, dependency, container scans
5. **PACKAGE** — Build container image and package artifacts
6. **PUBLISH** — Push artifacts / images to registry
7. **DEPLOY** — Deploy to target environment
8. **MONITOR** — Monitor app, pipeline, and user health

GOVERNANCE & COMPLIANCE

- Enforce policies with branch protection and required status checks.
- Audit all pipeline runs and approvals.
- Use least-privilege service accounts and environment protections.

LAB OVERVIEW -- GITHUB ACTIONS CI/CD PIPELINE

Ship a reviewable GitHub Actions workflow for the CRM: verify, optional scan, package-once identity, secrets hygiene, and a peer-usable runbook.

BUSINESS SCENARIO

- Pull requests need fast feedback while main and version tags require stronger gates -- verify always, package only after main/tags.
- Build output must be traceable and passed between isolated steps, or staging and production silently diverge from what CI actually verified.
- You own the CI contract for Labs 41-43: verify the CRM backend serving Amina (CUS-1001) and Ravi (CUS-1002), keep credentials out of Git, and leave evidence an on-call engineer can trust.

LEARNING OBJECTIVES

- Model CI stages and gates for PR, main, and version tags.
- Configure Maven dependency caching in GitHub Actions.
- Publish Surefire, Failsafe, and security-scan reports as artifacts.
- Use branch and pull-request pipelines with distinct rigor.
- Pass immutable artifacts (JAR + checksum + commit) between isolated steps.
- Document a reproducible re-run path in docs/ci-runbook.md.

WHAT YOU BUILD

- .github/workflows/ci.yml with a verify job (checkout, setup-java 21, Maven cache, clean verify, report upload) and a package job (needs: verify, package-once + SHA-256 checksum, gated to main/tags), plus docs/ci-runbook.md.

KEY TAKEAWAY Fixtures CUS-1001 (Amina) and CUS-1002 (Ravi) are synthetic -- never real customer data. Duration: about 45 minutes with starters, up to 3-4 hours full path.

GitHub Actions

LAB OVERVIEW – GITHUB ACTIONS CI/CD PIPELINE

In this lab, you will build a complete CI/CD pipeline using GitHub Actions for a full-stack Angular + Spring Boot application with PostgreSQL, and deploy it to OpenShift.

LAB OBJECTIVES

- Create a multi-stage GitHub Actions workflow.
- Build, test, and package backend (Spring Boot) and frontend (Angular).
- Run code quality and security scans.
- Build and publish Docker images.
- Deploy the application to OpenShift.
- Verify deployment and pipeline success.

LAB ENVIRONMENT

- GitHub repository provided
- GitHub Actions enabled
- Docker Hub / GitHub Container Registry
- OpenShift cluster access
- Required secrets and variables pre-configured

KEY DELIVERABLES

- ✓ Multi-stage GitHub Actions workflow (.yml)
- ✓ Successful builds and test reports
- ✓ Docker images published to registry
- ✓ Application deployed to OpenShift
- ✓ Pipeline execution summary

TECHNOLOGY STACK

Backend	Spring Boot, Maven, JUnit
Frontend	Angular, Node.js, Karma
Database	PostgreSQL
Container	Docker
Platform	OpenShift
CI/CD	GitHub Actions

CI/CD PIPELINE FLOW

1. CODE
Code commit & push

2. BUILD & TEST
- Backend build (Maven)
- Frontend build (npm)
- Run unit & integration tests

3. QUALITY & SECURITY
- CodeQL scan
- Dependency scan
- SAST checks

4. PACKAGE
- Build Docker images
- Upload artifacts

5. DEPLOY
- Push images to registry
- Deploy to OpenShift

6. VERIFY
- Health check
- Smoke test
- Pipeline success

PIPELINE STAGES IN THIS LAB

Stage	What You Will Do
1 Build & Test Backend	Compile Spring Boot application, run unit & integration tests.
2 Build & Test Frontend	Install dependencies, build Angular app, run tests.
3 Quality & Security	Run CodeQL, dependency vulnerability scan, and SAST checks.
4 Package & Publish	Build Docker images for backend and frontend. Push images to container registry.
5 Deploy to OpenShift	Deploy applications using manifests/Helm to OpenShift.
6 Verify Deployment	Perform health checks and smoke tests to validate success.

SUCCESS CRITERIA

- ✓ All pipeline stages complete successfully.
- ✓ Docker images available in the registry.
- ✓ Applications running on OpenShift.
- ✓ Health endpoints return 200 OK.
- ✓ Pipeline status shows green (success).

IMPORTANT
Use GitHub Secrets for sensitive data such as registry credentials and

LAB TASKS AND DELIVERABLES

lab43-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Define pipeline policy: PR/main/tag checks, deploy authority, retention rules.
2	Prepare reproducible commands: Maven Wrapper or pinned image, clean verify without skipTests.
3	Create the verify job: checkout, setup-java 21, Maven cache, upload Surefire/Failsafe reports.
4	Add a quality gate: dependency scan with a documented fail threshold.
5	Package once with a SHA-256 checksum tied to the commit; never rebuild in deploy.
6	Configure branch behavior: PR verify-only, main/tag package, manual-only deploy.
7	Protect variables: secured, environment-scoped secrets; names documented, never values.
8	Test, document, and force one controlled failure; restore green; write the runbook.

EXPECTED DELIVERABLES

- .github/workflows/ci.yml with PR / main / tag behavior
- Test reports (Surefire/Failsafe) evidence
- Security / scan report evidence (or documented residual risk)
- JAR and SHA-256 checksum tied to the commit
- docs/ci-runbook.md with policy, rerun, and troubleshooting

RUBRIC (100 MARKS)

Environment/Structure 10 * Core Pipeline 30 * Gates/Secrets 20 * Failure Handling 15 * Verification 15 * Docs 10

KEY TAKEAWAY -DskipTests on the default verify path is a named false-green anti-pattern. A deploy step that reruns mvn package breaks artifact identity. No secret value ever appears in YAML, logs, or screenshots.

GitHub Actions

LAB TASKS AND DELIVERABLES

In this lab, you will build a complete CI/CD pipeline using GitHub Actions for a full-stack Angular + Spring Boot application with PostgreSQL and deploy it to OpenShift.

LAB TASKS

1. **Prepare Repository**
   Fork or create the lab repository and review the project structure.
2. **Create Workflow**
   Create a GitHub Actions workflow YAML file in `.github/workflows/`.
3. **Build and Test Backend**
   Compile the Spring Boot application using Maven and run unit & integration tests.
4. **Build and Test Frontend**
   Install dependencies, run linting, unit tests, and build the Angular application.
5. **Code Quality and Security**
   Add CodeQL analysis and dependency vulnerability scanning.
6. **Build Docker Images**
   Create Dockerfiles for backend and frontend and build images.
7. **Push Images to Registry**
   Authenticate and push images to GitHub Container Registry (GHCR).
8. **Deploy to OpenShift**
   Deploy the application to OpenShift using Helm/manifest templates.
9. **Verify Deployment**
   Run smoke tests to verify application is accessible and healthy.
10. **Documentation**
    Document pipeline, architecture, and steps followed.

PIPELINE OVERVIEW

1. Code	2. Build & Test	3. Quality & Security	4. Package	5. Deploy	6. Verify
Commit & push code	Build app & run tests	CodeQL & dependency scans	Build Docker images	Deploy to OpenShift	Health check & smoke tests

KEY TECHNOLOGIES

LAB ENVIRONMENT

GitHub Repository	GitHub Container Registry (GHCR)	OpenShift Cluster	Secrets & Variables	Runners
Starter code provided	For storing Docker images	Dev namespace pre-provisioned	Pre-configured in repository	GitHub-hosted runners

DELIVERABLES

- [x] GitHub Actions workflow file (YAML)
- [x] Successful build and test reports
- [x] Code quality and security scan results
- [x] Docker images published to GHCR
- [x] Application deployed on OpenShift
- [x] Smoke test results
- [x] Pipeline execution summary
- [x] Documentation (steps, notes, screenshots)

EXPECTED OUTCOMES

- ✓ Automated CI/CD pipeline triggers on push.
- ✓ All stages complete successfully.
- ✓ Docker images are stored securely.
- ✓ Application is running on OpenShift.
- ✓ Pipeline status shows green (success).

LAB TIPS

- Use meaningful commit messages.
- Keep workflow steps small and modular.
- Use caching to speed up builds.
- Review logs carefully when failures occur.
- Iterate and improve your pipeline.

MODULE 43 SUMMARY

In this module, we built a reviewable GitHub Actions pipeline: triggers, jobs, caching, artifacts, quality gates, secrets, and a documented deployment path to OpenShift.

KEY CONCEPTS COVERED

Workflows & Triggers

on: push/pull_request/schedule/workflow_dispatch decide when a workflow runs at all.

Jobs, Steps & Runners

Jobs run on runners; steps run commands or actions; needs: orders jobs that must not run in parallel.

Caching & Artifacts

cache: maven/npm speeds builds; upload/download-artifact pass files between jobs and to humans.

Quality Gates

Tests, CodeQL, dependency scanning, and required status checks all block a bad merge.

Secrets & OIDC

Repository/environment secrets and OIDC keep credentials out of YAML and logs entirely.

Deployment to OpenShift

Package once, checksum it, then promote that exact artifact — never rebuild in the deploy job.

MODULE FLOW RECAP

1. Trigger & Verify
2. Cache & Test
3. Gate & Scan
4. Package Once
5. Deploy to OpenShift

KEY TAKEAWAY A trustworthy pipeline verifies on every PR, gates on main, packages once with a checksum, protects every secret, and documents exactly how a peer re-runs it.

GitHub Actions

MODULE SUMMARY

In this module, you learned how to design, build, and automate enterprise-ready CI/CD pipelines using GitHub Actions for a full-stack Angular + Spring Boot application and deploy to OpenShift.

KEY TAKEAWAY

Automate everything, validate continuously, and deliver with confidence.

WHAT YOU LEARNED

- Understood **CI/CD** concepts and the role of **GitHub Actions**.
- Created **workflows** using **events, jobs, steps, actions, and runners.**
- Built and tested **Java (Maven)** and **Angular** applications in GitHub Actions.
- Implemented **code quality, security scanning,** and dependency checks.
- Built and published **Docker** images to GitHub Container Registry **(GHCR).**
- Deployed applications to **OpenShift** using **Helm/manifests.**
- Used **secrets, variables, environments,** and **approval gates** for secure deployments.
- Monitored pipeline runs, diagnosed failures, and followed **enterprise best practices.**

CI/CD PIPELINE STAGES REVIEW

1. CODE	2. BUILD & TEST	3. QUALITY & SECURITY	4. PACKAGE	5. DEPLOY	6. VERIFY
Commit & push code	Build apps & run tests	Scan code & dependencies	Build Docker images	Deploy to OpenShift	Health & smoke tests

KEY COMPONENTS

Workflows Define automation in YAML	**Jobs & Steps** Organize tasks & execution order	**Actions** Reusable building blocks	**Runners** Execute workflows on infrastructure	**Secrets & Variables** Secure config & credentials
Containers Package apps consistently	**Container Registry** Store & manage Docker images	**OpenShift** Deploy & run applications	**Security Scanning** SAST, dependency, & container scans	**Monitoring** Logs, metrics & pipeline visibility

BUSINESS VALUE

- Faster delivery cycles with automated pipelines.
- Higher quality with continuous testing and security checks.
- Reliable releases with consistent and repeatable deployments.
- Improved visibility and control over the delivery process.
- Better collaboration with standardized workflows and documentation.

WHAT'S NEXT?

- Reuse workflows and create composite actions.
- Implement advanced strategies (Blue/Green, Canary).
- Integrate with more enterprise tools and systems.
- Continuously improve and optimize your pipelines.

BEST PRACTICES

- Automate everything and repeat manually
- Fail fast and provide clear feedback
- Keep workflows modular, maintainable, and reusable
- Secure your pipeline and protect sensitive
- Continuously monitor, measure, and improve

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of workflow structure, job ordering, and testing discipline.

1. Which top-level YAML keys are required in every GitHub Actions workflow file?

A. on and jobs only

B. name, on, and jobs

C. jobs and steps

D. runs-on and uses

2. What does needs: verify on a job called package actually do?

A. Nothing, it's a comment

B. Makes package wait for verify to succeed first

C. Runs package before verify

D. Deletes verify's artifacts

3. Why must -DskipTests never be used on the default verify path?

A. It speeds up the pipeline safely

B. It produces a false-green build that hides real failures

C. Maven requires it

D. It is required for caching to work

4. What is the correct response to a step echoing a secret's value into the log?

A. Ignore it, logs are private

B. Rotate the secret and remove the echo

C. Rename the secret

D. Add more logging for context

KEY TAKEAWAY name/on/jobs are required; needs: orders jobs; skipping tests for a green build is a named anti-pattern; leaked secrets get rotated immediately, not just renamed.

KNOWLEDGE CHECK

Let’s reinforce what you’ve learned in this module.

Instructions:

- Answer the following questions to **test your understanding** of key concepts.
- Some questions may have **one correct answer,** while others may have **multiple correct answers.**
- Review your answers and explanations to **strengthen your knowledge.**
- Use this as an opportunity to **identify areas of strength** and **topics to revisit.**

Every question is a step closer to mastery.
Let’s see how much you’ve learned!

KNOWLEDGE CHECK (2 OF 2)

Four more questions on caching, package-once identity, triggers, and OIDC.

5. What does cache: maven in actions/setup-java actually cache?

A. The JDK installation itself

B. ~/.m2/repository, keyed on pom.xml

C. The compiled JAR

D. GitHub's runner image

6. Why should a deploy job never run mvn package again?

A. It's slower than necessary

B. It breaks the chain of custody with what CI verified

C. Maven doesn't work in deploy jobs

D. It uses too much disk

7. Which trigger should NOT typically publish a checksummed deployable artifact?

A. push to main

B. a version tag (v*)

C. pull_request

D. release

8. What replaces a long-lived cloud credential stored as a GitHub secret when a provider supports it?

A. A hardcoded password in YAML

B. OIDC -- a short-lived, signed token requested per run

C. A personal access token in a comment

D. Nothing, secrets are always required

KEY TAKEAWAY Maven caching targets ~/.m2 keyed on pom.xml; deploy never rebuilds; PRs don't publish deployable artifacts; OIDC replaces long-lived deploy secrets wherever supported.

"A pipeline that quietly skips tests is not green — it is blind."

MODULE 43 COMPLETE

GitHub Actions and CI/CD Integration

You can now design a GitHub Actions workflow — triggers, jobs, caching, artifacts, quality gates, secrets, and OIDC — and ship a package-once, checksummed build of the CRM to OpenShift.